

PAM Platform

Architecture, Authentication, Decisions, Roadmap & Hardening

Generated 2026-05-11 · Platform hardening complete

Contents

1. Architecture
2. Core Platform Mapping
3. Authentication & Authorization
4. Decisions (ADRs)
5. Roadmap
6. Local Development
7. Platform Hardening
8. Testing

Architecture

This document captures the **why** behind the architectural decisions for the PAM (Player Admin Manager). What the code does is in the code; this document exists for the decisions a future engineer can't recover from reading source.

Modular monolith, not microservices (yet)

We build as a single deployable that is **internally** structured like a set of services. Each module lives under `src/Modules/<Module>/Pam.<Module>/` with its own `DbContext`, schema (`<module>.*`), aggregates, and feature folders.

- A module can be extracted to its own service when scale or team-org demands it. The module boundary (assembly, schema, contracts) makes that mechanical.
- Until then, we get a single deployable, one Postgres backup, one connection pool, and zero distributed-system tax.

Hard rule: a module never reads another module's data or types directly. The only way modules communicate is:

1. **Integration events** through MassTransit + RabbitMQ (default).
2. **In-process queries** through `IQuery<T>` interfaces in `Pam.<Module>.Contracts`.
3. Never via `DbContext` or direct project reference into `Pam.<Module>` from another module.

The contracts assembly is the seam. A future microservice extraction is "replace the in-process implementation behind `IBrandLookup` with a gRPC client" — same interface, different transport.

Per-module pattern (established Phase 2)

Every module follows the same shape. `Pam.Operators` is the reference; copy its scaffolding for module #2 and beyond.

Three projects per module:

- `Pam.<Module>.Contracts` — DTOs, `IQuery<T>` definitions, integration events. The only thing other modules and `Pam.Api` reference for cross-module reads.
- `Pam.<Module>` — aggregates, features (vertical slices), domain events, EF `DbContext` + migrations, module wire-up. Internal to the module.
- `tests/Pam.<Module>.UnitTests` — aggregate behavior + validator tests. No DB, no broker.

Folder shape inside `Pam.<Module>`:

```

<AggregateName>/
Models/<Aggregate>.cs
Features/<UseCase>/
  <UseCase>Command.cs
  <UseCase>Validator.cs
  <UseCase>Handler.cs
  <UseCase>Endpoint.cs
Events/<DomainEvent>.cs
EventHandlers/<DomainEvent>DomainHandler.cs # bridges to integration event
Exceptions/<Aggregate>Errors.cs
Data/
  <Module>DbContext.cs
  <Module>DbContextDesignTimeFactory.cs
Configurations/<Aggregate>Configuration.cs
Migrations/
<Module>Module.cs # AddXModule + UseXModuleAsync

```

Module wire-up. Static `<Module>Module` class with two extension methods:

- `AddXModule(this IServiceCollection, IConfiguration)` — registers `DbContext`, interceptors, health checks.
- `UseXModuleAsync(this(IServiceProvider)` — runs `db.Database.MigrateAsync()` on startup.

`AuditableSaveChangesInterceptor` and `DispatchDomainEventsInterceptor` (from `Pam.Shared`) are registered per-module via `services.TryAddScoped<...>()` so duplicate registrations across modules are idempotent.

Persistence rules:

- Schema-per-module via `modelBuilder.HasDefaultSchema("<module>")` and `npg.MigrationsHistoryTable("__EFMigrationsHistory", "<module>")`.
- Snake_case columns via the `EFCore.NamingConventions` package (`options.UseSnakeCaseNamingConvention()`). Apply on **both** the runtime `DbContext` options and the design-time factory, or `dotnet ef migrations add scaffolds PascalCase` column names.
- Enum columns persisted as string with `.HasConversion<string>().HasMaxLength(N)`. Keeps the column self-describing and makes adding values append-only at any ordinal position.
- Audit columns (`created_at`, `created_by_type`, `created_by_id`, `last_modified_*`) configured explicitly on every entity.

Health check: `services.AddHealthChecks().AddNpgsql(connectionString, name: "<module>-db", tags: ["ready", "module:<module>"])`.

Identity: embedded OpenIddict + ASP.NET Core Identity

PAM owns its identity stack. There is no external IDP process.

- **OpenIddict** (MIT, embedded as a NuGet package) issues OAuth 2.0 / OIDC tokens. Authorization Code + PKCE for the back-office SPA, refresh tokens with rotation, `client_credentials` pre-enabled for future module-to-module calls.
- **ASP.NET Core Identity** owns user storage, password hashing, lockout, email confirmation, MFA. Lives in the `Pam.Identity` module.
- **Single host, single Postgres database.** Identity tables and OpenIddict tables cohabit `IdentityDbContext` in schema `identity`.

- **Roles + Permissions.** Identity provides `IdentityRole<Guid>` for coarse buckets (Owner, Manager, Operator, Accountant). On top of that, a `Permission + RolePermission` join carries fine-grained codes (`operators.read`, `players.write`, ...). Both project as claims at token-issuance time; both can be referenced from `[Authorize(Policy = ...)]`.

This is the "we own the entire code" answer. The tradeoff vs an external IDP (ZITADEL/Keycloak) is more code we maintain in exchange for: no separate process, no separate DB, no Brand ↔ Org sync layer, and any breaking change hits us only on a deliberate package upgrade. See ADR #16.

`Pam.Identity` lands in Phase 3. Until then, every endpoint is anonymous — intentional.

Brand is a first-class aggregate in `Pam.Operators`

Multi-brand is a first-class concern: betanything.eu plus the planned LATAM and Asia expansions all share this PAM.

- `Brand` (in `Pam.Operators`) owns name, slug (URL-safe identifier, unique per platform), jurisdiction (regulatory region), status (Active / Suspended / Disabled), and the audit columns. More fields land as the module grows (default currency, default language, brand metadata, jurisdictional policy refs).
- Every brand-scoped aggregate (Player, Wallet, Bonus, ...) carries a `BrandId` foreign reference.
- Brand uniqueness is enforced at the DB (`UNIQUE (slug)`) and via an in-handler precheck so we can return a typed `AlreadyExistsException` (→ 409) instead of a Postgres unique-violation generic 500.
- Brand resolution at runtime: anonymous registration takes brand context from a request header (final shape TBD when Players returns); back-office identities carry `brand_id` as a JWT claim issued by the embedded IDP.

When more brand-scoped modules ship, each `DbContext` will gain an EF Core global query filter on `BrandId` to prevent the "forgot to filter by tenant" data-leak class of bug.

Lean integration events, no PII

Integration events carry IDs and routing data only — never email, name, DOB, or anything else regulators would consider PII. Consumers that need richer data go through the relevant `Pam.<Module>.Contracts` query (e.g. `GetBrandByIdQuery`) to fetch it.

The reason is that integration events live in the message broker, the outbox table, audit logs, and any consumer that listens. Putting PII into them fans out PII to all of those stores and makes GDPR right-to-be-forgotten a nightmare. Lean events are the regulated-finance default.

Error model: `ProblemDetails` + stable string codes

All errors are RFC 7807 `ProblemDetails`. Domain exceptions carry a stable string error code (`operators.brand.slug-taken`, `operators.brand.not-found`, ...) under `extensions.code`. Clients program against the code; messages can be localized and tweaked freely.

Validation errors flow through `FluentValidation` and become `ValidationProblemDetails` with a per-field error array.

Every response includes a `traceId` (W3C Activity ID) for support correlation.

Time

`DateTimeOffset` everywhere, stored as `timestampz`. `IClock` interface (with `SystemClock` impl) is injected wherever a timestamp matters, so handlers and domain code are testable.

`DateTime.Now`, `DateTime.UtcNow`, and `DateTimeOffset.Now` are banned via `BannedSymbols.txt`. `DateTimeOffset.UtcNow` is allowed (intentional — used by the `IntegrationEvent` base record's init defaults and similar places where DI is impractical).

Audit columns

`Entity<TId>` carries `CreatedAt`, `CreatedByType`, `CreatedById`, `LastModifiedAt`, `LastModifiedByType`, `LastModifiedById`. The `AuditableSaveChangesInterceptor` stamps these on every save using `IClock` and `IUserContext`.

`IUserContext.Current` returns a typed `Actor(ActorType, Id)` — never null, defaulting to `Actor.System` for background work and `Actor.Anonymous` for unauthenticated requests. `ActorType` $\in$ {`System`, `Player`, `Operator`, `Service`, `Anonymous`}. The Phase 3 `Pam.Identity` `CurrentUserService` will populate this from JWT claims (user id + a discriminator derived from the issuing audience or scope set).

This is the regulator-checkable discriminator: a `Player` self-suspending vs an `Operator` suspending them are distinguishable in the audit columns without parsing.

This is application audit logging, not regulatory audit logging. Regulatory audit (immutable, cryptographically chained) is a separate `Pam.Audits` module that subscribes to integration events. It will arrive when there's a real regulatory requirement to satisfy.

Domain events vs integration events

Two distinct concerns:

Domain event	Integration event
Inside one module	Across modules
<code>Pam.<Module>/<Concept>/Events/</code>	<code>Pam.<Module>.Contracts/<Concept>/IntegrationEvents/</code>
Internal — refactor freely	Public contract — versioned, breaking changes are real
Dispatched in-process by <code>DispatchDomainEventsInterceptor</code> (post-save)	Outbox + MassTransit (when consumers arrive)
Naming: <code>BrandCreated</code>	Naming: <code>BrandCreatedIntegrationEvent</code>

Domain events do not implement MediatR's `INotification`. They implement our own `IDomainEvent`. A wrapper (`DomainEventNotification<TEvent>`) adapts them for MediatR dispatch. This keeps the domain free of framework types.

A handler inside the module bridges the two: e.g. `BrandCreatedDomainHandler` listens to `BrandCreatedDomainEvent` (in `Pam.Operators`) and publishes `BrandCreatedIntegrationEvent` (from `Pam.Operators.Contracts`) via MassTransit's `IPublishEndpoint`. The integration event preserves the original `EventId` and `OccurredAt`.

Integration events describe what happened, not what to do

Single most important rule for shaping an integration event. The publisher broadcasts a fact (`UserLoggedIn` , `PlayerRegistered` , `BetSettled`). Subscribers decide what to do with it.

The trap is publishing a "command-shaped" event:

```
// WRONG – this is RPC dressed up as an event
public sealed record SendEmailIntegrationEvent(
    string To, string Subject, string Body
) : IntegrationEvent;
```

That couples the publisher to:

- the recipient's email + locale (whose data?)
- the email body wording (whose template?)
- whether to send at all (whose policy?)
- per-brand branding (where does it live?)

...and it leaks PII through every queue and audit log it touches.

```
// RIGHT – describe the fact, leave the action to subscribers
public sealed record UserLoggedInIntegrationEvent(
    Guid UserId,
    string IPAddress,
    string UserAgent,
    string? DeviceFingerprint
) : IntegrationEvent;
```

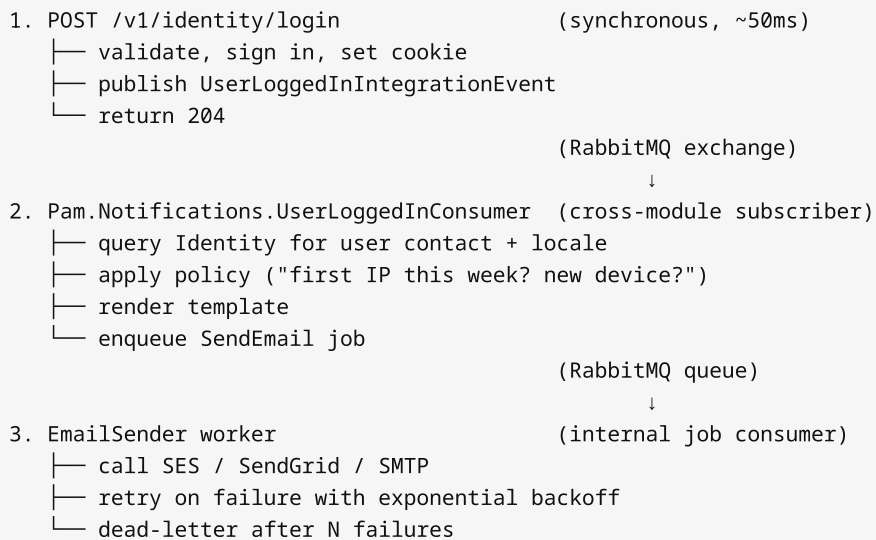
`Pam.Notifications` (when it ships) subscribes, queries Identity for the user's contact + locale, decides if the login warrants an email, renders the template, and only *then* enqueues a send-email job. The publisher stays ignorant of all that.

Integration events vs job queues — both ride RabbitMQ

Two different patterns, both transported by MassTransit/RabbitMQ.

	Integration event	Job queue
Question	"What happened?"	"When can this work run?"
Producer	Notifies the world	Schedules work
Consumers	Many — fan-out broadcast	One worker per job
Caller waits?	No (fire-and-forget)	Sometimes (poll for status)
Retry/scheduling?	Optional	Core feature
Coupling	Decouples <i>modules</i> from each other	Decouples <i>time of request</i> from <i>time of work</i>

A typical flow chains them:



Step 1 is synchronous — the user is waiting. Step 2 is the integration event (decouple modules). Step 3 is the job queue (decouple work from time, with retries). Three concerns, one broker.

What gets queued vs what stays synchronous

Rule of thumb: anything where the user is waiting for the result of a *decision* runs synchronously. Anything that's a side effect of that decision runs async.

Operation	Pattern	Why
POST /login	Synchronous	User waiting; cookie in response
POST /register	Synchronous	Validation must reject immediately
POST /reset-password	Synchronous	Validation + token; returns 200
Sending the welcome email	Queued	SMTP can fail; needs retries
Sending an SMS via Twilio	Queued	Rate-limited downstream API
Webhook to a game provider	Queued	Retry until 200 or dead-letter
Bulk-importing players from gbs-db	Queued	Long-running
Generating a daily revenue report	Queued	Heavy aggregation
Real-time bet placement	Synchronous	<50ms latency budget

Returning 202 Accepted with a job id is occasionally the right call — mostly for genuinely long-running operations the user explicitly initiated (a CSV import). It's almost never the right call for a user action with a yes/no decision behind it.

Notifications and cross-module email

`Pam.Notifications` is the only module that owns an SMTP gateway (`ISmtpEmailSender` + the MailKit-backed `SmtpEmailSender`). The interface ships in `Pam.Notifications.Contracts` ; every other module that needs to send mail injects it from there. Two paths into the gateway, used for different shapes of need:

Path 1 — direct `ISender.SendAsync(...)`

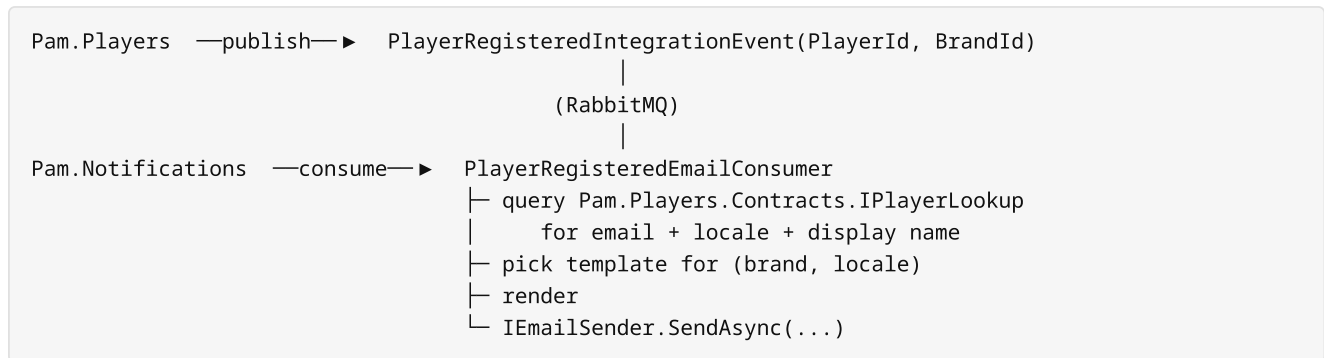
Use this when the **publisher owns the content** AND **the payload is sensitive enough that putting it on the broker is a bad idea**.

Concrete example: `Pam.Identity`'s `ForgotPasswordHandler` calls `ISender` directly. The handler has the user, generates the reset token, embeds it in the link, and sends — all inside one transaction. The token IS the credential. Putting it in a `PasswordResetRequested-IntegrationEvent` would fan it out to every consumer's logs, the outbox table, the audit trail, and any future regulatory-audit subscriber. That isn't acceptable for a thing that grants access to an account.

The other intra-module sends in Identity (email confirmation, MFA admin reset notifications when those land) follow the same pattern.

Path 2 — publish a fact-shaped integration event

Use this for **cross-module** flows where the originating module *shouldn't* know the template, the recipient's locale, the brand styling, or even whether an email is the right channel.



The event is lean (just IDs). The consumer in `Pam.Notifications/ Consumers/` decides whether to send, picks the template, renders it, and calls `ISender`. This is the right shape for: Players welcome email, Wallet deposit confirmation, Bonuses awarded, KYC verified, suspicious-login alerts, etc.

Why split this way

If every module imported `ISender` and inlined templates, you'd end up with:

- Email body wording scattered across N modules.
- Per-brand branding duplicated everywhere.
- Locale resolution duplicated everywhere.
- Compliance review needing to audit N modules instead of 1.

Funneling cross-cutting "we observed X, the user should probably be told" through integration events keeps all template + branding + delivery policy in one place — `Pam.Notifications`. The originating module just describes what happened.

The two-path split is the same architectural principle that makes integration events fact-shaped (see *Integration events describe what happened, not what to do* above). The direct-call path is the carved-out exception for "the publisher genuinely does own the content because it owns the secret."

Aggregate sizing rules

Three rules, repeat in code review:

1. One command modifies one aggregate in one transaction. If a command needs two, the boundary is wrong or you need a saga.
2. Reference other aggregates by ID, never by navigation property.
3. Default to small aggregates. Merge only when an invariant requires atomic writes across them.

Secrets

Two-layer model:

1. **Non-secret config** (URLs, log levels, feature flags) lives in `appsettings.{env}.json`, committed.
2. **Secrets** (DB connection strings, RabbitMQ passwords, OpenIddict signing/encryption keys, payment-provider HMACs) arrive as environment variables from the host. ASP.NET's default configuration precedence reads env vars over `appsettings.{env}.json` and treats `__` as the nesting separator (`ConnectionStrings__Pam` → `ConnectionStrings:Pam`).

In production those env vars are injected by whatever orchestrator runs the API — systemd unit, k3s Secret, Swarm secret. The orchestrator-managed secret model is sufficient for the current deployment shape; a dedicated secret store (HashiCorp Vault, SOPS, k3s External Secrets, ...) is an open question pinned in `ROADMAP.md` and will be evaluated when the first real production deploy lands.

Outbox + pre-save domain-event dispatch

Domain events dispatch **pre-save**: `DispatchDomainEventsInterceptor` overrides `SavingChangesAsync` so handlers run inside the same DB transaction as the `SaveChanges` that triggered them. The dispatch loop is bounded at 8 generations to catch handlers that recursively raise events on tracked aggregates.

MassTransit's EF Core outbox is wired on `OperatorsDbContext` (and any future publishing module's `DbContext`). `BrandCreatedDomainHandler` calls `IPublishEndpoint.Publish(BrandCreatedIntegrationEvent)`; the outbox intercepts that publish and writes the message to `outbox_message` in the same transaction. A background `EntityFrameworkOutboxDeliveryService` polls the table and forwards to RabbitMQ. DB write + queued integration event commit atomically; a crash between commit and delivery is harmless because the delivery service retries until the row is marked sent.

Trade-off you're buying into:

- Handlers see **pre-commit** state. The aggregate row isn't visible to other connections yet, so a handler that queries the DB for freshly-written data will miss it. Workaround: pass the data through the event payload.
- A throwing handler **rolls back the whole `SaveChanges`**. That's the point — atomic by design. Don't put best-effort side effects (logging, metrics) in a domain handler; put them in an integration-event consumer.

Per-module outbox wiring. Each publishing module exposes a public

`ConfigureOutbox(IBusRegistrationConfigurator) delegate` (see `OperatorsModule`); `Program.cs` passes those delegates to `AddPamMassTransit`. Adding a new publisher = three lines: package reference, model entities (`AddInboxStateEntity` / `AddOutboxMessageEntity` / `AddOutboxStateEntity`), `ConfigureOutbox` delegate.

For Wallet the outbox is non-negotiable from day one of that module — the wallet's `LedgerEntryPosted` integration event **MUST** be transactional with the ledger row. This pattern is the substrate that makes it possible.

Core Platform → PAM mapping

This is the feature-parity reference plus the "10x better" thesis. The full manual is at `~/Downloads/Core Platform.pdf` (68 pages, end-user BO doc).

What Core Platform is

A multi-tenant iGaming back-office (sportsbook + casino + bet shops). The manual is end-user oriented — it describes what an operator sees in the BO UI, not the underlying API. The functional surface is enormous (~20 top-level sections, hundreds of entities, embedded website builder, embedded CRM, embedded affiliate panel).

The system is **partner-scoped** throughout — every entity belongs to a "Partner" (the white-label brand). Partners own their own product catalog, currencies, languages, payment providers, website fragments, banners, translations, and limits.

Section → module mapping

Core Platform section	PAM module	Status
Partners (Main / Payment / Product / Currency / Language / Provider / Limits / Country / Comp Points / Payment Info / Keys)	Pam.Operators (we use Brand , not Partner)	v1 (Brand CRUD) — Pam.Operators shipped Phase 2 with the Brand aggregate. More partner-scoped fields land as sub-aggregates.
Users & Agents + Roles	Pam.Identity (back-office; OpenIddict + ASP.NET Identity)	Phase 3 — in flight. Embedded IDP, roles + permissions.
Clients > Main / Notes / Settings	Pam.Players	deferred (returns as module #3, post-Identity)
Clients > Sessions	Pam.Players (Sessions sub-aggregate)	deferred
Clients > KYC	Pam.Kyc (likely starts inside Pam.Players)	deferred
Clients > Limits & Exclusions / Product Limits	Pam.Limits	deferred
Clients > Corrections / Deposits / Withdrawals / Account History / Transactions / Payment Info / Payment Settings	Pam.Wallet	deferred (most regulated)
Clients > Campaigns / Bets	Pam.Bonuses + Pam.Bets	deferred
Clients > Tickets	Pam.Support	deferred
Clients > Friends	Pam.Affiliates (referrals)	deferred
Clients > Emails & SMSes / Provider Settings	Pam.Notifications + Pam.Players	deferred
Online Clients (Real Time)	streaming projection over Pam.Players sessions	deferred
Affiliates + Affiliate admin panel	Pam.Affiliates (separate module + own auth/admin UI)	deferred
Payments (global)	Pam.Wallet query side	deferred
Bets / Internet Bets / Bet Shops	Pam.Bets + Pam.BetShops	deferred
Bonuses (Common + Triggers + Conditions DSL)	Pam.Bonuses	deferred (own design pass)
Segments	Pam.Segments (cross-cutting query/projection)	deferred
Partners > Website Settings (fragments / banners / image bars / sport widgets / footer / casino / virtual / translations / styles)	Pam.Cms (heavy)	deferred
Partners > Backup & Restore	event-sourced state per module	covered by event sourcing approach

Core Platform section	PAM module	Status
Bet Shops / Providers / Product Categories / Products	Pam.GameCatalog	deferred
Accounting (bet shop monthly)	Pam.Reporting (read-side projection)	deferred
Reports (Bets / Clients / Documents / Providers / Identity)	Pam.Reporting	deferred
Notifications (sent log)	Pam.Notifications	deferred
Currencies	reference data in Pam.Operators	deferred
CRM Templates	Pam.Notifications (templates)	deferred
CMS (Banners / Promotions / Regions / Comment Types / Job Areas / Translations / Enumerations / Security Questions / Online Shop)	Pam.Cms + Pam.Shop	deferred
(Implicit) Game wallet integration with providers	Pam.GameWallet (separate host)	deferred

Decisions to make NOW to enable "10x better"

These are the architectural calls that are cheap if made before scaling out the modules and very expensive afterwards. They're load-bearing for the whole product, not for any one module.

1. Multi-brand is a first-class aggregate, not a column

Core Platform's "Partner" is everywhere but feels retrofitted. We bake it in from day one — under the term **Brand** (we're a single company running multiple consumer-facing brands; "Partner" is a B2B term that doesn't fit):

- A `Brand` aggregate in `Pam.Operators` is the first module — shipped in Phase 2.
- Every other aggregate carries a `BrandId` foreign reference.
- The same physical human registering on Brand A and Brand B is two distinct `Player` rows (per-brand records); cross-brand identity linking is a future optional `Subject` concept, introduced only when regulation demands it.
- Brand context for a back-office actor rides as a `brand_id` claim in the JWT issued by the embedded IDP (`Pam.Identity`, Phase 3); for anonymous flows (player registration, when Players returns), brand context comes from a request header.
- Each `DbContext` will gain a global query filter on `BrandId` so cross-brand reads are impossible by default once back-office endpoints arrive.
- Operator endpoints can elevate to cross-brand access via a specific permission (`operator.platform-admin`).

Cost of doing this in v2: rewriting every query in every module. Cost of doing it now: one filter per DbContext. Easy choice.

2. Wallet is double-entry from day one

Core Platform exposes "Used Balance / Unused Balance / Bonus Balance / Comp Balance / Coin Balance" — five ad-hoc balances with no clear consistency story. The "Corrections" section lets operators directly debit/credit any balance.

We replace this with a **double-entry ledger**:

- One `LedgerEntry` aggregate (immutable, append-only).
- `Account` types: `cash`, `bonus`, `loyalty_comp`, `loyalty_coin`, `held_for_bet`, `held_for_withdrawal`, etc.
- A `Wallet` projection that aggregates entries into balances per account type. Snapshots every N entries for performance.
- "Corrections" become specific posting types in the ledger (with an operator actor and a reason code), not arbitrary balance writes.
- Outbox-published `LedgerPosted` integration events power Reporting, Audit, Notifications.

Reason: regulators require immutable transaction history. Ad-hoc balance-mutation is the #1 cause of wallet incidents in iGaming. Modeling it correctly once is dramatically cheaper than retrofitting.

3. Bonus engine: typed DSL, not enum-driven hardcoded conditions

Core Platform's bonus engine is a `BonusConditionTypes` enum (Sport, Region, Match, Selection, Stake, etc.) crossed with an `OperationTypes` enum (IsEqualTo, InSet, AtLeastOneInSet, ...). It's a proprietary mini-DSL embedded in their database.

Better: use a **typed expression language** (e.g. CEL / NRules / a small custom Specification pattern) that:

- Compiles to typed predicates against domain events.
- Evaluates in real time by subscribing to integration events (`BetPlaced`, `BetSettled`, `DepositCompleted`, `LoginCompleted`).
- Has a designer UI in the BO that produces the same expression tree.
- Can be unit-tested standalone.

Bonus types from the manual (Campaign Wager Sport/Casino, Cash, Free Bet, Free Spin, Cashback, Affiliate, Raffle, Tournament, Challenges) become specific configurations of the same engine, not separate code paths.

4. Audit is event-sourced and immutable, not "Changes History" notes

Core Platform has a "Changes History" panel on each entity showing who edited what. That's app-level audit, fine for ops. **Regulatory** audit needs to be append-only and tamper-evident.

`Pam.Audits` (event-sourced, no UPDATES) subscribes to every integration event and stores (`occurredAt`, `actor`, `ip`, `correlationId`, `payload`, `hash`, `prevHash`). Hashes chain so any tampering is detectable. Regulators ask; this answers.

5. API-first with stable contracts

Core Platform is BO-first — the BO is the application. We invert: the API is the application, and there are multiple consumers (BO UI, mobile app, public website, partner integrations, game-wallet ingress).

Practically:

- Every module exposes its public surface as `Pam.<Module>.Contracts` (already established).
- OpenAPI spec is the source of truth, exposed via Scalar.
- Typed clients (NSwag-generated) for BO UI.

- Versioning from day one (/v1/ . . .) so contract evolution is explicit.

6. Modern auth replaces "Force Block at 5 wrong passwords"

Core Platform's logout is primitive: 5 wrong passwords → "Force Block" that requires support intervention. Their state machine is Active/Blocked/Disabled/Force Block/Suspended.

We have **embedded OpenIddict + ASP.NET Core Identity** (`Pam.Identity` , Phase 3) driving:

- Configurable brute-force protection (Identity logout: 5 attempts → 5 min lockout out of the box; tunable).
- MFA (TOTP via Identity authenticators; WebAuthn via OpenIddict + Fido2 package when the trigger arrives).
- Refresh-token rotation with replay detection (OpenIddict's authorization storage; on by default — strongly discouraged to disable).
- Session/token revocation tied to self-exclusion: `Limits` emits `SelfExclusionActivated` → `Pam.Identity` revokes the user's tokens via OpenIddict's authorization-entry validation hook (`EnableAuthorizationEntryValidation()`).
- Federation for operators (external OIDC providers via OpenIddict's client/validation stack — Azure AD, Google Workspace) when the back-office team requires it.
- Concurrent session limits, session listing, "log out everywhere."

Their five player states map onto our richer state machine + the IDP's session/credential state, with each transition emitting a domain event that Audit consumes.

7. Real-time via streaming, not polling

Core Platform has "Online Clients (Real Time)" and a Dashboard with metrics. Their UX implies polling.

We push:

- Integration events fan out via MassTransit.
- A SignalR (or native WebSocket) hub in `Pam.Api` subscribes to relevant events and pushes to BO clients.
- The BO Dashboard is a live projection of `LoginCompleted` , `DepositCompleted` , `BetSettled` , etc.

8. Typed config, not JSON-in-BO

Core Platform's "Web Fragments" / "Style Type" / "Banner Settings" all have free-form JSON inputs in the BO. This is brittle (typos break the site, no validation).

We define typed schemas per fragment kind (`BannerFragment` , `ImageBarFragment` , `SportWidgetFragment` ...) with versioning and JSON-schema-driven validation in the API. The BO renders a typed form per kind, not a free-text JSON box.

9. CMS is its own bounded context with snapshotting

Core Platform's website builder (Banners + Promotions + Web Fragments + Translations + Styles + Image Bars + Sport Widgets + Footer per partner) is genuinely a CMS. Treat it as one:

- `Pam.Cms` module with its own schema.
- Snapshots: a partner's entire website config can be cloned, branched, rolled back. (Their "Backup & Restore for Website Settings" feature becomes free with event sourcing.)
- Preview environment: a partner sees their changes on a staging domain before publishing.

10. Localization at the API layer

Core Platform stores translations per-partner per-section in the BO and operators must manually upload. We do:

- Translatable resources are first-class with a `LocalizedString` value object ({en: "...", es: "...", ...}).
- Default language fallback chain.
- Translation management via Crowdin/Lokalise integration so non-technical staff can manage translations outside the BO.

Build order, revised given the full feature surface

Ordered by what unblocks the most subsequent modules:

1. **Pam.Operators (Brand aggregate) — shipped Phase 2.** Multi-tenant foundation; every later aggregate carries `BrandId`.
2. **Pam.Identity (OpenIdDict + ASP.NET Core Identity) — Phase 3, in flight.** Back-office Users & Agents, roles + permissions. Without this no human can act on Brand or any future aggregate.
3. **Pam.Players** (re-introduce as module #3) — Player aggregate scoped under a Brand, authenticated via the embedded IDP. KYC, sessions, limits land as sub-aggregates initially.
4. **Pam.Wallet (double-entry ledger) —** the most regulated module. Outbox is non-negotiable here. Brand + Identity + Players are prerequisites.
5. **Pam.GameProviders / Pam.GameCatalog** — enables real bets to flow.
6. **Pam.GameWallet (separate host) —** game-provider HMAC-signed ingress, sub-200ms p99. Calls `Pam.Wallet` via contracts.
7. **Pam.Bets + Pam.Bonuses (with the typed DSL design) —** depends on Wallet + GameCatalog.
8. **Pam.Notifications + Pam.Audits** — subscribing modules; cheap once integration events exist.
9. **Pam.Affiliates** — own admin panel, commissions, referral tracking.
10. **Pam.Cms + Pam.Shop** — biggest UI surface; can be built late.
11. **Pam.Reporting** — read-side projections over events; can be incrementally added.

The current focus is items 2-3 (Identity + Players return). Item 4 (Wallet) is the next major milestone after that.

Authentication and Authorization

End-to-end reference for `Pam.Identity` — the back-office auth module. This doc covers what's actually shipped today (Phase 3, three PRs) and the contracts other modules consume.

Scope. Back-office Users & Agents only. The Players module returns in Phase 4 and will use the *same* OpenIddict server under a different audience (`pam_player_api`). Until then, every `/v1/*` endpoint that isn't `.AllowAnonymous()` requires a back-office token.

The stack

Layer	What it does	Where it lives
OpenIddict	OAuth 2.0 / OIDC token issuance + validation. AuthorizationCode + PKCE + Refresh.	<code>IdentityModule.cs</code> (<code>AddOpenIddict().AddCore/AddServer/AddValidation</code>)
ASP.NET Core Identity	User storage, password hashing, logout, TOTP.	<code>BackOfficeUser : IdentityUser<Guid></code> , <code>IdentityDbContext</code>
Cookie auth	The IDP session. Set on <code>POST /v1/identity/login</code> , read by <code>/connect/authorize</code> .	<code>ConfigureApplicationCookie</code> in <code>IdentityModule.cs</code>
Bearer tokens	API auth. Sent as <code>Authorization: Bearer <access_token></code> on <code>/v1/*</code> .	<code>OpenIddictValidation...</code> scheme in <code>Program.cs</code>
Per-permission policies	Fine-grained authz. One policy per code in <code>PermissionCodes</code> .	<code>AddAuthorization</code> in <code>Program.cs</code>
HttpContext	Translates the OIDC <code>sub</code> to an <code>Actor</code> for audit columns.	<code>Pam.Identity/Authentication/HttpContext.cs</code>

Single host, single database. Identity tables, OpenIddict tables, and the custom permissions table all cohabit `IdentityDbContext` in schema `identity`. No separate IDP process, no separate DB.

See [DECISIONS.md #16](#) for why this beats an external IDP.

DB layout (schema `identity`)

<code>users</code>	<code>BackOfficeUser : IdentityUser<Guid> + BrandId + audit</code>
<code>roles</code>	<code>IdentityRole<Guid></code>
<code>user_roles</code>	<code>user ↔ role (M:N)</code>
<code>user_claims, role_claims</code>	per-user / per-role claims (unused today)
<code>user_logins</code>	external SSO links (unused today, see Federation)
<code>user_tokens</code>	TOTP secret + "remember this machine" cookie pin
<code>permissions</code>	custom – string codes from <code>PermissionCodes</code>
<code>role_permissions</code>	<code>role ↔ permission (M:N)</code>
<code>openidict_applications</code>	OAuth clients (pam-bo today)
<code>openidict_scopes</code>	pam_api + standard OIDC scopes
<code>openidict_authorizations</code>	one row per (user, client, scopes) grant
<code>openidict_tokens</code>	every access / refresh / authcode ever issued

The Quartz cleanup job (`UseQuartz()` in `AddCore`) prunes expired and revoked tokens hourly, so `openidict_tokens` tracks active sessions, not all-time history.

Roles + permissions

Two layers of authz, both project as claims at token-issuance time.

Roles (coarse, seeded at startup):

Role	Default permissions
Owner	All of them, including <code>platform.admin</code>
Manager	<code>operators.brands.*</code> , <code>identity.users.*</code> , <code>identity.roles.write</code>
Operator	<code>operators.brands.read</code> , <code>identity.users.read</code>
Accountant	<code>operators.brands.read</code>

Permissions (fine-grained, declared in `Pam.Identity.Contracts.Permissions.PermissionCodes`):

```
operators.brands.read
operators.brands.write
identity.users.read
identity.users.write
identity.roles.write
platform.admin          ← meta: grants every other policy via assertion
```

Endpoints gate on permissions, not roles:

```
app.MapPost("/v1/identity/users", ...)
    .RequireAuthorization($"Permissions.{PermissionCodes.Identity.UsersWrite}");
```

`platform.admin` is an Owner-only escape hatch — the policy authorization handler accepts the request if the user has the specific permission **OR** `platform.admin`. See [DECISIONS.md #20](#).

Flows

Bootstrap — first run

Env vars seed the very first Owner so somebody can log in:

```
PAM_BOOTSTRAP_OWNER_EMAIL=owner@you.local  
PAM_BOOTSTRAP_OWNER_PASSWORD=<initial>
```

`IdentitySeeder.SeedBootstrapOwnerAsync` runs at API startup:

1. If any user with the `Owner` role already exists → skip.
2. Else, if either env var is missing → log a warning and skip (the API still boots, but no one can log in until you provision an Owner).
3. Else, create the user with `EmailConfirmed = true` and assign the `Owner` role.

After the first Owner exists, the env vars are ignored on subsequent restarts. Every other back-office user is created via `POST /v1/identity/users`. There is **no self-service back-office registration** — that's a privilege-boundary breach. See ROADMAP.

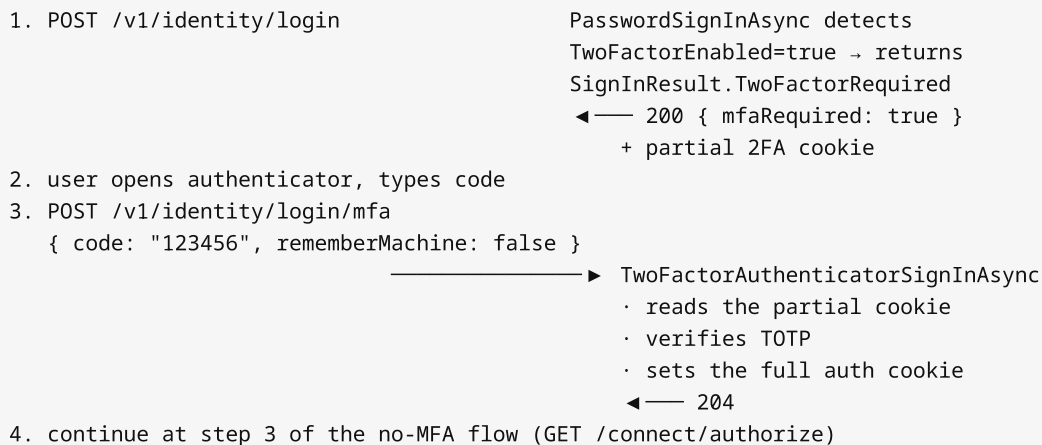
Login — no MFA



The cookie is set in step 2 and persists for the entire IDP session. The access token is in SPA memory and lives ~1 hour; the refresh token lives ~14 days. See [Token lifecycle](#).

Login — with MFA (TOTP)

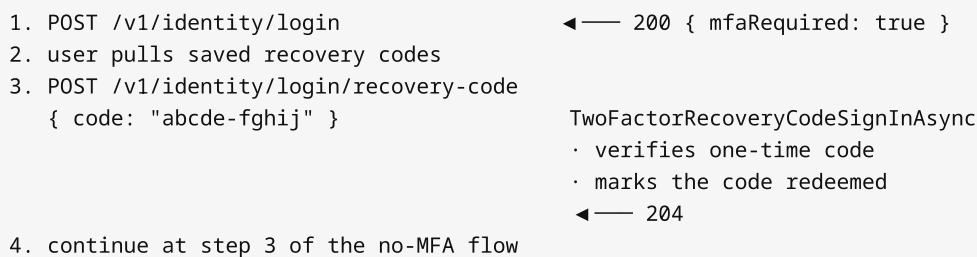
User had previously enrolled an authenticator. Login bifurcates after the password check:



`rememberMachine: true` writes a long-lived "two-factor-remembered" cookie that bypasses MFA on this browser for the next login. Tracked in `user_tokens` (provider `[AspNetUserStore]`, name `RememberClient`).

Login — with a recovery code

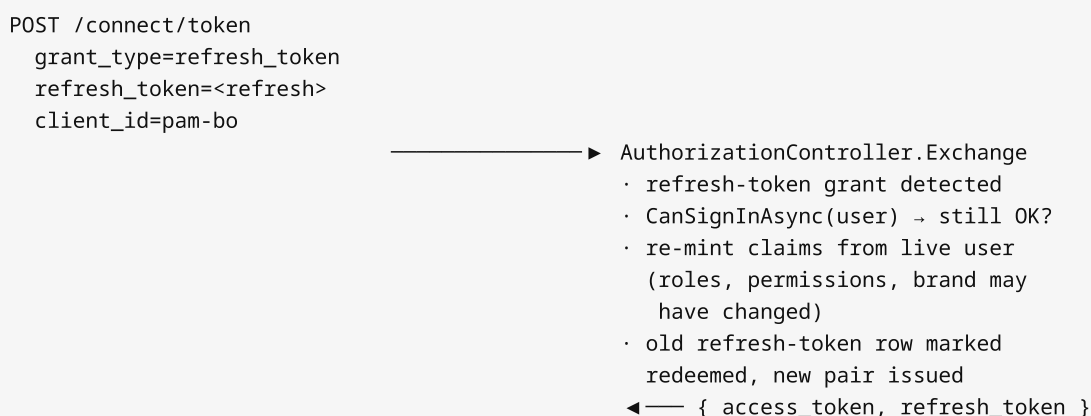
User lost their authenticator. Same partial-cookie flow:



Each recovery code is **one-time use** — Identity stores them hashed and marks them as consumed on use.

Token refresh (silent renewal)

The SPA refreshes its access token in the background before the current one expires:



Refresh-token rotation is on by default. Each refresh invalidates the previous refresh token, so replaying a captured refresh token after the user has refreshed once gets the attacker rejected.

Logout

Three things you might want to mean by "logout":

Want	Endpoint(s)	What it does
End the cookie session	GET POST /connect/logout	SignInManager.SignOutAsync() clears the auth cookie + the OIDC id_token_hint flow redirects to post_logout_redirect_uri. Existing access tokens still work for their TTL.
Kill the refresh token	POST /connect/revocation (token=refresh)	Marks the refresh row revoked. Combined with EnableTokenEntryValidation, the access token issued from it fails at the next API call.
Kill everything for this user	Admin action: DELETE /v1/identity/users/{id} or POST /v1/identity/users/{id}/mfa/reset	Rotates the security stamp + revokes outstanding authorizations. Instant when entry validation is on.

SPA logout should call **both** /connect/logout and /connect/revocation (with the refresh token) for proper hygiene.

Self-service: change password

```

POST /v1/identity/me/change-password
{ currentPassword, newPassword }    [authenticated, rate-limited 5/min/sub]
                                   ───────────────────────────────────────────▶ ChangePasswordHandler
                                   · UserManager.ChangePasswordAsync
                                   · security stamp auto-rotated
                                   ◀── 204

```

CurrentPassword is required even when authenticated — a captured session cookie shouldn't be able to silently change the password.

Self-service: forgot / reset password

```

1. POST /v1/identity/forgot-password    [anonymous, rate-limited]
   { email }
                                   ───────────────────────────────────────────▶ ForgotPasswordHandler
                                   · find user by email
                                   · IF user exists AND email
                                     is confirmed:
                                     - generate reset token
                                     - email link
                                   · ALWAYS 204
                                   ◀── 204

2. user clicks link in email
   GET https://spa/reset-password?email=...&token=...

3. SPA shows new-password form

4. POST /v1/identity/reset-password    [anonymous, rate-limited]
   { email, token, newPassword }
                                   ───────────────────────────────────────────▶ ResetPasswordHandler
                                   · UserManager.ResetPasswordAsync
                                   · security stamp rotated
                                   ◀── 204 or 422

```

Anti-enumeration design — /forgot-password always returns 204, even for unknown emails or unconfirmed accounts. SMTP failures are caught + logged so the response shape stays constant under outage; the email

address is **not** logged (that's the PII this endpoint protects). See [ForgotPasswordHandler](#).

Email confirmation

A new admin-created user starts with `EmailConfirmed = false`. The confirmation flow:

```
admin: POST /v1/identity/users      —▶ CreateUserHandler
                                     · UserManager.CreateAsync
                                     · AddToRolesAsync
                                     · AUTO-FIRE SendConfirmationEmailCommand
                                       (logged-only on SMTP failure;
                                       admin can retry below)
                                     ◀— 201
                                     — email sent —▶ (user's inbox)

user clicks link from email:
GET https://spa/confirm-email?email=...&token=...
SPA: POST /v1/identity/confirm-email ◀— { email, token }
                                     · UserManager.ConfirmEmailAsync
                                     ◀— 204

admin retry (if email lost / SMTP was down):
POST /v1/identity/users/{id}/send-confirmation-email [identity.users.write]
                                     · idempotent – no-op if already confirmed
                                     ◀— 204
```

Gotcha for production. `options.SignIn.RequireConfirmedEmail = false` is currently set in `IdentityModule.cs`. That means a created- but-unconfirmed user CAN log in. To make confirmation mandatory before login, flip it to `true`. The bootstrap Owner is fine either way (seeded with `EmailConfirmed = true`).

MFA — enroll

```
POST /v1/identity/me/mfa/enroll      [authenticated]
                                     —▶ MfaEnrollHandler
                                     · GetAuthenticatorKeyAsync (or
                                       ResetAuthenticatorKeyAsync
                                       if first time)
                                     · build otpauth:// URI
                                     ◀— { sharedKey, authenticatorUri }

SPA renders the URI as a QR code or shows the
sharedKey for manual entry. User scans into
Authenticator app, gets a 6-digit code.

POST /v1/identity/me/mfa/verify      [authenticated, rate-limited]
{ code: "123456" }                  —▶ MfaVerifyHandler
                                     · VerifyTwoFactorTokenAsync
                                     · SetTwoFactorEnabledAsync(true)
                                     ◀— 204
```

The authenticator key persists until `SetTwoFactorEnabledAsync(true)` succeeds — re-calling `/enroll` returns the same key, not a fresh one. That's intentional: the user might close the QR modal before scanning and need to come back. Only after the user explicitly disables MFA and re-enrolls does a new key get generated.

MFA — recovery codes

```
POST /v1/identity/me/mfa/recovery-codes      [authenticated, rate-limited]
  (requires TwoFactorEnabled=true)
  ───────────────────────────────────────────▶ GenerateRecoveryCodesHandler
                                              · GenerateNewTwoFactorRecoveryCodesAsync(user, 10)
                                              ◀── { codes: [10 codes] }
```

Plaintext returned **once**. Codes are stored hashed in `user_tokens`. Re-issuing invalidates the previous batch.

MFA — disable (self)

```
POST /v1/identity/me/mfa/disable              [authenticated, rate-limited]
  { currentPassword }
  ───────────────────────────────────────────▶ MfaDisableHandler
                                              · CheckPasswordAsync (challenge)
                                              · SetTwoFactorEnabledAsync(false)
                                              · ResetAuthenticatorKeyAsync
                                              (so re-enable starts fresh)
                                              ◀── 204
```

`CurrentPassword` again — same reason as change-password. A captured session can't weaken auth posture.

MFA — admin reset

For "user lost their phone AND their recovery codes." Hard escape hatch:

```
POST /v1/identity/users/{id}/mfa/reset        [identity.users.write]
  ───────────────────────────────────────────▶ ResetMfaHandler
                                              · SetTwoFactorEnabledAsync(false)
                                              · ResetAuthenticatorKeyAsync
                                              · UpdateSecurityStampAsync
                                              → outstanding tokens die
                                              ◀── 204
```

After reset, the user logs in with password only (TwoFactor is off) and can re-enroll via `/me/mfa/enroll`.

Admin operations

All under `/v1/identity/users`, all gated on `identity.users.write` or `identity.users.read`.

Method	Path	Permission	Purpose
POST	/v1/identity/users	identity.users.write	Create user. Auto-fires confirmation email.
GET	/v1/identity/users? page=&pageSize=&brandId=&role=&lockedOut=	identity.users.read	Paged list with filters.
GET	/v1/identity/users/{id}	identity.users.read	Detail.
PATCH	/v1/identity/users/{id}	identity.users.write	Email / BrandId / LockoutEnabled. null = no change.
DELETE	/v1/identity/users/{id}	identity.users.write	Soft-delete (LockoutEnd = MaxValue + security-stamp rotation).
POST	/v1/identity/users/{id}/roles	identity.roles.write	Assign role. Idempotent.
DELETE	/v1/identity/users/{id}/roles/{role}	identity.roles.write	Remove role. Idempotent.
POST	/v1/identity/users/{id}/unlock	identity.users.write	Clear failed-attempt lockout. Refuses on soft-deleted users.
POST	/v1/identity/users/{id}/mfa/reset	identity.users.write	Admin MFA escape hatch.
POST	/v1/identity/users/{id}/send-confirmation-email	identity.users.write	Re-send confirmation email. Idempotent.

Why soft-delete via lockout? Regulatory retention forbids hard delete (the user's actions must stay auditable).

LockoutEnd = DateTimeOffset.MaxValue rides Identity's existing lockout infrastructure:

SignInManager.PasswordSignInAsync already refuses to sign locked-out users in.

Why role changes rotate the security stamp? So tokens issued before the change re-validate within the security-stamp interval and pick up the new claim set. Combined with EnableAuthorizationEntryValidation, the effect is ~immediate.

Token lifecycle

Token	Default TTL	Where it lives	Refresh on use?
Authorization code	5 minutes	openidict_tokens (single-use)	n/a
Access token	1 hour	SPA memory	No — get a new one via refresh
Refresh token	14 days	SPA memory + openidict_tokens	Yes — rotation on every refresh
Auth cookie (IDP session)	8 hours sliding	Browser, HttpOnly	Yes — sliding window
2FA "remember machine"	14 days	Browser + user_tokens	No

Entry validation — what it buys

`AddValidation()` in `IdentityModule.cs` is configured with:

```
options.EnableAuthorizationEntryValidation();
options.EnableTokenEntryValidation();
```

Effect: every API call cross-checks the access token's row in `openiddict_tokens` and its parent authorization row in `openiddict_authorizations`. If either has status `revoked` or `inactive`, the call 401s. **Revocation is therefore effectively instant** — kill the row, every subsequent call dies.

Cost: 2 extra Postgres lookups per authenticated request. Both are PK reads on indexed columns; sub-ms each; Postgres caches them in shared buffers. For the back-office surface (tens to low-hundreds of operators) this is fine. For the future `Pam.GameWallet` host (sub-200ms p99, thousands of req/s) it would not be — that host should run its own validation stack **without** these flags and accept short revocation lag. See the comments in [IdentityModule.cs](#).

How tokens die

Trigger	Mechanism	Latency
Token TTL expires	OpenIddict validation rejects	At expiry
<code>/connect/revocation</code> called	Token row → revoked	Next API call (entry validation)
User soft-deleted	Security stamp rotates	Next security-stamp check (~30min default for cookie; immediate for tokens via entry validation)
Role / permission changed	Security stamp rotates	Same as above
Password reset	Security stamp rotates	Same

Cookie vs token

After a successful login the SPA has two parallel auth mechanisms:

	Identity cookie	Bearer token
Set by	Server, on <code>POST /v1/identity/login</code> (<code>Set-Cookie</code>)	Returned in <code>POST /connect/token</code> response body
Stored where	Browser, <code>HttpOnly</code> (SPA JS can't read)	SPA memory
Sent to	<code>/connect/*</code> endpoints (automatic, browser attaches)	<code>/v1/*</code> endpoints (SPA attaches <code>Authorization: Bearer</code>)
Purpose	Identifies user TO THE IDP	Authorizes API calls
Lifetime	8h sliding (or browser-close, depending on <code>Remember Me</code>)	1h access / 14d refresh
Survives browser restart?	Only if <code>rememberMe=true</code> on login	Tokens are in memory — gone on tab close

So the SPA's actual "I'm still logged in" state is held by the bearer tokens. The cookie matters when the SPA needs to refresh tokens silently via `/connect/authorize` (no UI prompt because cookie still proves identity).

"Remember Me" vs "Remember Machine"

Two different flags on two different endpoints:

Flag	Endpoint	Effect
<code>rememberMe: true</code>	<code>/v1/identity/login</code>	Auth cookie becomes persistent (survives browser restart). Does not affect tokens or MFA.
<code>rememberMachine: true</code>	<code>/v1/identity/login/mfa</code>	Writes the 2FA-remembered cookie. This browser bypasses MFA on the next login. Independent of <code>rememberMe</code> .

Configuration

All keys live under `appsettings.{env}.json` (or env vars with `__` nesting: `Identity__BackOfficeSpa__LoginUrl=...`).

Identity:BackOfficeSpa (BackOfficeSpaOptions)

Key	Default	Purpose
<code>ClientId</code>	<code>pam-bo</code>	OpenIddict client id seeded in <code>openiddict_applications</code> .
<code>DisplayName</code>	<code>PAM Back-Office SPA</code>	Cosmetic.
<code>LoginUrl</code>	<code>http://localhost:3000/login</code>	Where the cookie middleware redirects on <code>/connect/authorize</code> without a session. Adds <code>?returnUrl=...</code> .
<code>ResetPasswordUrl</code>	<code>http://localhost:3000/reset-password</code>	Where <code>/forgot-password</code> emails point.
<code>ConfirmEmailUrl</code>	<code>http://localhost:3000/confirm-email</code>	Where confirmation emails point.
<code>RedirectUris</code>	<code>[http://localhost:3000/auth/callback]</code>	OIDC <code>redirect_uri</code> allowlist.
<code>PostLogoutRedirectUris</code>	<code>[http://localhost:3000/]</code>	OIDC <code>post_logout_redirect_uri</code> allowlist.

Notifications:Smtp (SmtpOptions , in Pam.Notifications)

Key	Default	Production setting
Host	localhost	Your SMTP relay (SES, SendGrid, ...)
Port	1025 (Mailpit)	587 for SES STARTTLS
UseStartTls	false	true
Username	(none)	provider credential
Password	(none)	provider credential — env var only
FromAddress	no-reply@pam.local	A real domain you control
FromName	PAM	Brand-aware in the future

Bootstrap env vars

Var	When
PAM_BOOTSTRAP_OWNER_EMAIL	First run only, while no Owner exists
PAM_BOOTSTRAP_OWNER_PASSWORD	First run only, while no Owner exists

Hardcoded policies (IdentityModule.cs)

Setting	Value	How to change
Password min length	12	Edit <code>options.Password.RequiredLength</code>
Password complexity	digit + lower + upper + nonalphanumeric, 4 unique chars	Edit <code>options.Password.*</code>
Lockout after	5 failed attempts	<code>options.Lockout.MaxFailedAccessAttempts</code>
Lockout duration	15 min	<code>options.Lockout.DefaultLockoutTimeSpan</code>
Cookie sliding expiration	8h	<code>options.ExpireTimeSpan</code>
<code>RequireConfirmedEmail</code>	false	Flip to <code>true</code> in prod (and after seeding all existing users to <code>EmailConfirmed=true</code>).
<code>RequireConfirmedAccount</code>	false	Stricter than above; rarely needed if <code>RequireConfirmedEmail=true</code> .

Per-environment knobs

`AddIdentityModule(builder.Configuration, builder.Environment)` checks `environment.IsDevelopment()` and disables OpenIddict's HTTPS requirement only in dev:

```
if (environment.IsDevelopment())
{
    aspNetCore.DisableTransportSecurityRequirement();
}
```

In prod the TLS-terminating reverse proxy / ingress sets `X-Forwarded-Proto: https` ;
`app.UseForwardedHeaders()` makes `Request.IsHttps` report true; OpenIddict accepts.

Dev signing/encryption certs

`AddDevelopmentEncryptionCertificate()` + `AddDevelopmentSigningCertificate()` generate self-signed certs into the user's profile store on first run and reuse them across restarts. **Production must replace these** with real certs from the host (separate from the HTTPS cert; OpenIddict recommends 2048-bit RSA, rotated annually). Pinned as TBD in ROADMAP.

What's NOT implemented (intentional)

Capability	Why deferred	Trigger to revisit
Self-service back-office registration	Wrong threat model — privilege boundary breach. Bootstrap Owner is the on-ramp.	Never.
Dynamic client registration (<code>/connect/register</code>)	We own every client; <code>pam-bo</code> today, future module-to-module clients seeded.	Never — third party app needing to register.
WebAuthn / passkeys	TOTP covers MFA sufficiently for the back-office threat model.	A regulator or customer asks. Code path: OpenIddict + Fido2 package.
External OIDC federation (Azure AD, Google Workspace)	No customer asking; back-office is small.	Customer or compliance team requires single-sign-on. <code>user_logins</code> table is already there.
Active sessions list / "logout everywhere" UI	Not a regulatory requirement today. Token revocation works via admin endpoints + <code>/connect/revocation</code> .	Operator UX feature request.
Concurrent session limits	Same reason.	Same trigger.
Player auth (<code>POST /v1/auth/register</code> etc.)	Players module deferred to Phase 4. Will use the same OpenIddict server under a different audience (<code>pam_player_api</code>).	The Players module itself.

Implementation map

Where each piece lives

```
src/Modules/Identity/Pam.Identity/
  IdentityModule.cs          ← AddIdentityModule wire-up, OpenIddict setup
  BackOfficeSpaOptions.cs    ← strongly-typed SPA config

  Data/
    IdentityDbContext.cs     ← all identity.* tables
    IdentityDbContextDesignTimeFactory.cs
    Configurations/          ← EF Core fluent config
    Migrations/              ← EF Core migrations

  Users/
    Models/BackOfficeUser.cs ← IdentityUser<Guid> + BrandId + audit
    Exceptions/UserErrors.cs ← stable error codes
    Features/
      CreateUser/            ← POST /v1/identity/users
      ListUsers/             ← GET /v1/identity/users
      GetUser/               ← GET /v1/identity/users/{id}
      UpdateUser/            ← PATCH /v1/identity/users/{id}
      DeleteUser/            ← DELETE /v1/identity/users/{id} (soft)
      AssignRole/            ← POST /v1/identity/users/{id}/roles
      RemoveRole/            ← DELETE /v1/identity/users/{id}/roles/{role}
      UnlockUser/            ← POST /v1/identity/users/{id}/unlock
      ResetMfa/              ← POST /v1/identity/users/{id}/mfa/reset
      SendConfirmationEmail/ ← POST /v1/identity/users/{id}/send-confirmation-email

  Authentication/
    HttpUserContext.cs       ← reads sub claim → Actor for audit columns
    Login/                   ← POST /v1/identity/login
    LoginMfa/                ← POST /v1/identity/login/mfa
    LoginRecoveryCode/       ← POST /v1/identity/login/recovery-code
    ChangePassword/          ← POST /v1/identity/me/change-password
    ForgotPassword/          ← POST /v1/identity/forgot-password
    ResetPassword/           ← POST /v1/identity/reset-password
    ConfirmEmail/            ← POST /v1/identity/confirm-email
    Mfa/                     ← /me/mfa/enroll, /verify, /disable, /recovery-codes

  OpenIddict/
    ClaimDestinations.cs     ← maps claims → access-token vs id-token
    Controllers/
      AuthorizationController.cs ← /connect/authorize + /connect/token + /connect/logout
      UserinfoController.cs      ← /connect/userinfo

  Permissions/
    Models/Permission.cs     ← entity
    Models/RolePermission.cs ← entity
    PermissionResolver.cs    ← role[] → permission code[]

  Seeding/
    IdentitySeeder.cs        ← permissions, roles, role-permissions,
                                scopes, applications, bootstrap Owner

src/Modules/Identity/Pam.Identity.Contracts/
  Permissions/PermissionCodes.cs ← well-known permission strings
  Permissions/PamClaimTypes.cs  ← custom claim type names
```

```

Roles/RoleNames.cs           ← well-known role names
Users/Dtos/BackOfficeUserDto.cs ← public response shape

src/Modules/Notifications/Pam.Notifications.Contracts/
Email/IEmailSender.cs         ← cross-module email surface
Email/EmailMessage.cs         ← DTO

src/Modules/Notifications/Pam.Notifications/
Email/SmtpEmailSender.cs      ← MailKit impl (internal)
Email/SmtpOptions.cs          ← Notifications:Smtp config
NotificationsModule.cs        ← AddNotificationsModule
Consumers/                    ← future integration-event subscribers

```

Patterns to copy when adding a new auth-adjacent endpoint

1. New feature under `Pam.Identity/<area>/Features/<UseCase>/`.
2. Four files: `<UseCase>Command.cs`, `<UseCase>Validator.cs`, `<UseCase>Handler.cs`, `<UseCase>Endpoint.cs`.
3. Endpoint either `.RequireAuthorization($"Permissions.{code}")` for admin actions, or `.RequireAuthorization()` for "any authenticated back-office user", or `.AllowAnonymous()` for public flows.
4. Anonymous flows (`/login`, `/forgot-password`, `/reset-password`, `/confirm-email`, `/login/mfa`, `/login/recovery-code`) MUST `.RequireRateLimiting("auth-sensitive")`.
5. Authenticated self-service that exposes secrets or weakens auth posture (change-password, mfa/disable, mfa/verify, mfa/recovery-codes) SHOULD `.RequireRateLimiting("auth-sensitive")` too.
6. Stable error codes go in `Users/Exceptions/UserErrors.cs` (or a sibling file in your area).
7. For email sending: inject `IEmailSender` from `Pam.Notifications.Contracts`. For sensitive content (tokens), call it directly. For "describe what happened" cross-module flows, publish an integration event and let a `Pam.Notifications/Consumers/` subscriber render the email.

Smoke testing the full flow

A bash script that exercises register → login → token → API call lives in the team folklore but isn't checked in yet. The bones:

```

# 1. Make sure the bootstrap Owner exists (env vars set in launchSettings.json
#    or shell before `make dev-api`).

# 2. Cookie-based login
curl -i -c jar -X POST http://localhost:5000/v1/identity/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"owner@test.local","password":"OwnerPassword123!"}'
# → 204 + Set-Cookie

# 3. userinfo with the cookie
curl -i -b jar http://localhost:5000/connect/userinfo
# → 200 + { sub, email, name, role: [Owner] }

# 4. Bearer-token via OIDC code flow needs PKCE – see the "What you can
#    do with the cookie" section in this doc's git history, or use
#    Scalar with OAuth configured.

```

See [LOCAL_DEV.md](#) for the rest of the dev loop.

Decisions

A log of architectural decisions made during the project's life. Each entry captures the **context** (why the choice came up), the **decision**, and the **consequences** (what we accept by choosing this). Entries are append-only — supersede rather than rewrite when a decision changes.

Rough ADR shape, kept terse. Order is reverse-chronological.

20. Roles + Permissions for back-office authz, not roles only — 2026-05-07

Context. The peer `pro-cr-billing` project uses three role-based policies (Owner / Accountant / Operator) with no permission layer. The PAM operator manual ("Employee Access and Security") and the legacy `IqSoft.CP.BLL.PermissionBll` both demand fine-grained per-action permissions. Retrofitting permissions across endpoints once dozens are in place is painful.

Decision. Phase 3 `Pam.Identity` ships with both:

- `IdentityRole<Guid>` (Owner / Manager / Operator / Accountant — coarse buckets, seeded at startup).
- A custom `Permission` + `RolePermission` join (well-known string codes like `operators.read`, `players.write`).

Both project as claims at token-issuance time via `SetDestinations . [Authorize(Policy = "Permissions.Players.Read")]` and `RequireRole(...)` are both supported; choose the granularity each endpoint actually needs.

Consequences.

- Slightly more code at module start. Big payoff when a customer-support team needs to do X but not Y.
 - Permission codes live as constants in `Pam.Identity.Contracts`; other modules reference those constants (no string typos).
-

19. Per-module 3-project pattern with Add+Use extension methods — 2026-05-07

Context. Without a written rule, module shape drifts. Some modules might grow Application/Domain/Infrastructure layers, others go vertical slice, others mix. Future readers and `ultrareview`-style architecture checks need a single template.

Decision. Every module ships as exactly three projects:

- `Pam.<Module>.Contracts` — DTOs, `IQuery<T>`, integration events.
- `Pam.<Module>` — aggregates, vertical-slice features, EF, wire-up.
- `tests/Pam.<Module>.UnitTests` — aggregate + validator unit tests.

Wire-up via a static `<Module>Module` class exposing `AddXModule(IServiceCollection, IConfiguration)` and `UseXModuleAsync(IServiceProvider)`. Schema-per-module + snake_case naming + explicit health probe. Interceptors registered per-module via `TryAddScoped` so multiple modules don't conflict.

Consequences.

- New-module cost is low; copy `Pam.Operators` and rename.
 - `NetArchTest.Rules` (pinned in catalog, wiring deferred) can enforce the boundary contract once a second module exists.
 - `Pam.Operators` is the canonical reference. Diverging from this pattern requires a new ADR.
-

18. Snake_case columns via EFCore.NamingConventions — 2026-05-07

Context. EF Core defaults to PascalCase column names matching C# property names. PostgreSQL idiom is snake_case; the peer `pro-cr-billing` project uses snake_case throughout. Inconsistency between case styles in the same project is a smell, and retrofitting once N modules ship is N migrations.

Decision. Add `EFCore.NamingConventions` 10.0.0 to the central catalog. Every module `DbContext` applies `options.UseSnakeCaseNamingConvention()` on **both** the runtime DI registration and the design-time factory. Without it on the design-time side, `dotnet ef migrations add` scaffolds PascalCase column names that disagree with runtime queries.

Consequences.

- DB tables read naturally from `psql`. Column names match Postgres conventions and tooling.
 - Established before Phase 3 to avoid renaming Identity + OpenIddict schemas later.
-

17. Operators-first module ordering; Players deferred — 2026-05-06

Context. The original sequencing was Players first (with a hardcoded `IBrandRegistry`), then Operators promotion, then KYC/Wallet/etc. After the ZITADEL drop (#16), the dependency graph looked wrong: back-office Identity is needed before any module can be admin-managed, and Brand is the multi-tenant foundation that every other aggregate references. Putting Players first meant baking in a brand abstraction layer twice (once as a hardcoded shim, once as the real thing) and rewriting the existing Player-Brand wiring later.

Decision. New module sequence: **Operators** → **Identity** → **Players** → **Wallet** → **KYC** → **Limits** → **Bonuses** → **Bets** → **Affiliates** → **Notifications** → **Reporting** → **Audit**. Phase 1 deleted the existing `Pam.Players` module and its tests entirely; Phase 2 shipped `Pam.Operators` with the `Brand` aggregate; Phase 3 is `Pam.Identity` (back-office Users & Agents). Players gets re-introduced as the fourth module, now properly scoped under a `Brand` and authenticated via the embedded IDP.

Consequences.

- ~2200 lines of Players + ZITADEL scaffolding deleted (commit `2a90022`); not lost — every pattern worth keeping was either already in `Pam.Shared` or is being rebuilt cleanly.
 - Memory file `project_phase1_pivot.md` captures the rationale and the patterns the next phase inherits.
-

16. Drop ZITADEL; embed OpenIddict + ASP.NET Core Identity — 2026-05-06

Context. Decisions #4 and #12 picked Keycloak then ZITADEL as the external IDP. The company's stronger requirement is sovereignty over the auth code path — "we want to own the entire codebase." External IDPs introduce: (a) a separate process to operate, (b) a second DB to back up and migrate, (c) a release cadence we don't control, (d) cross-process domain leakage (`Brand` ↔ `ZITADEL Org` sync).

Decision. Replace ZITADEL with **OpenIddict + ASP.NET Core Identity**, both embedded as NuGet packages in `Pam.Identity` (Phase 3).

- OpenIddict (MIT) issues OAuth 2.0 / OIDC tokens. Authorization Code + PKCE + Refresh for the back-office SPA; client_credentials pre-enabled for future module extraction.
- ASP.NET Core Identity owns user storage, password hashing, lockout, email confirmation, MFA. Both stacks share IdentityDbContext in schema identity.
- BackOfficeUser : IdentityUser<Guid> carries BrandId. Player authentication is deferred to when the Players module returns; design intent is a distinct flow within the same OpenIddict server (likely a separate scope set).
- Phase 1 deletes everything ZITADEL: infra/zitadel/, the three ZITADEL services from docker-compose.yml, all Zitadel.* package refs, the IIdentityProvider ZITADEL implementation, BrandRegistry, IBrandRegistry, and the related Players-module scaffolding. Phase 3 brings auth back via OpenIddict.

Consequences.

- Single process, single DB, MIT/Apache-2.0 source we ship in our binary. If the maintainer disappears we keep building.
- More code we own (custom /connect/authorize + /connect/token controllers; permissions seeder; cert rotation). Bounded; documented in project_phase1_pivot.md.
- Supersedes #4, #12, #15.

15. ZITADEL bootstrap as IHostedService, not a shell script — 2026-05-06

Status: superseded by #16.

Context. Decision #12 introduced ZITADEL with a bash + curl + python3 bootstrap script that read the admin PAT from a Docker volume, then made three API calls to ensure Orgs and Project. The script kept hitting portability issues: volume-path lookup differs per Docker setup, ZITADEL changed the org-search endpoint between versions, the chicken-and-egg PAT extraction needed permission fix-ups, and idempotency-check fallbacks were noisy on fresh runs.

Decision. Remove infra/zitadel/bootstrap.sh and .env.zitadel. Bootstrap moves into Pam.Api as ZitadelBootstrapService : IHostedService, which:

- waits for ZITADEL's OIDC discovery endpoint to come online (2-min cap),
- reads the PAT from a configurable file path (default infra/zitadel/machinekey/zitadel-admin-sa.pat, walks up to repo root if invoked from a sub-dir),
- ensures the configured Brand Orgs exist (gRPC OrganizationService.AddOrganization via the smartive Zitadel SDK; on AlreadyExists falls back to ListOrganizations),
- ensures the Project exists in the default brand's Org,
- writes the resulting Org IDs and an ITokenProvider (built from the PAT) into ZitadelRuntimeState (singleton). BrandRegistry reads the org map; ZitadelClientFactory reads the token provider to mint per-org gRPC management clients on demand.

Consequences.

- One language. No python3 / curl / bash shell-quoting fragility.
- API fails fast at startup if ZITADEL isn't reachable — the same property we want for any IDP-dependent service.
- mprocs goes from three procs (services / bootstrap / api) to two (services / api), matching the peer pro-cr-billing pattern.

- `BrandRegistryOptions.Map` collapses to `BrandRegistryOptions.BrandIds` (a list) — Org IDs are runtime-resolved, not config-supplied.
 - `ZitadelOptions.AdminPat` becomes `ZitadelOptions.AdminPatFile`.
-

14. Per-brand Player records; defer cross-brand `Subject` — 2026-05-06

Context. With multi-brand confirmed, the question is whether a single human registering on Brand A and Brand B should be one PAM record or two. The legacy production system uses pattern (a) — one row regardless of brand — which conflates a person with an account: shared wallet across brands (illegal in many jurisdictions), one brand's compliance officer sees other brands' data, conflicting per-jurisdiction RG limits on a single row.

Decision. Each `(person × brand)` is its own `Player` row. Email uniqueness is per-brand (composite unique index on `(brand_id, email)`). The legacy pattern is **not** carried forward. A future optional `Subject / Person` concept can link `Player`s across brands when a specific regulatory requirement arrives (UK GamStop-style central self-exclusion, FATF travel-rule recognition); until then, "same human re-verifies KYC on each brand" is the accepted cost.

Consequences.

- Wallet, Limits, Bonus, KYC are per-Player by construction.
 - Cross-brand fraud detection requires a future `Subject` layer; deferred until a real regulator asks.
 - The legacy data migration story is "split, not preserve": each `(person, brand)` pair in the legacy `tbCustomer` becomes a separate `Player` row, with a stub `Subject` linking them only if mandated.
-

13. Brand is first-class; one ZITADEL Org per Brand — 2026-05-06

Context. `betanything.eu` is one company today, planning expansion to LATAM and Asia with crypto. That is multi-brand (multiple consumer-facing sites under one operator) and multi-jurisdiction. Adding a `BrandId` to every aggregate after KYC, Wallet, Limits, Bonus all ship is much more expensive than adding it before module #2.

Decision. A `Brand` concept is first-class from day one. Every aggregate carries `BrandId`. ZITADEL Org per Brand: registration sets `x-zitadel-orgid` on Management API calls; the `IIdentityProvider` impl resolves `Brand → Org` via an injected `IBrandRegistry`. Anonymous registration takes brand context from the `X-Brand` header (default `betanything-eu`); authenticated requests will derive brand from the player record itself. Email uniqueness is per-brand (`(brand_id, email)` composite unique index).

The full `Pam.Operators` module (Brand + Jurisdiction policy registry, admin endpoints) is deferred to before module #2. Today the registry is a hardcoded `IBrandRegistry` reading from `appsettings:Brands:Map`, populated by the bootstrap script's `.env.zitadel`.

Consequences.

- POC seeds two Orgs (`betanything-eu`, `betanything-latam-stub`) so the multi-brand path is exercised, not assumed.
 - Ahead of jurisdictional rules: `Jurisdiction` stays a value object on `Player` for now; a `Pam.Operators` jurisdiction policy module (min age, KYC tier matrix, allowed currencies) lands when KYC does.
 - ZITADEL's tenant boundary aligns with the brand boundary by design; no per-brand realm-import dance.
-

12. ZITADEL replaces Keycloak — 2026-05-06

Context. Decision #4 deferred a ZITADEL spike to "the operators realm milestone." Multi-brand confirmation (decision #13) shifted that trigger forward — committing to a Brand model is the moment to question the IDP shape, because ZITADEL's Org/Project model maps naturally to Brand and Keycloak's realm-per-brand pattern multiplies operational pain (N realm imports, N admin clients, N token endpoints).

Decision. Replace Keycloak end-to-end with ZITADEL. Brand → Org, single Project (pam-player-api) audience for the player API, machine PAT for Management API access. The `IIIdentityProvider` seam is preserved; `KeycloakIdentityProvider` is replaced with `ZitadelIdentityProvider`. The `player_id` claim is dropped from JWTs entirely — the IDP `sub` is the foreign key on the Player aggregate, and PAM resolves `PlayerId` server-side via that lookup. Tokens stay lean.

Consequences.

- One Postgres-backed self-host service replaces the JVM-heavy Keycloak
 - Keycloak-Postgres pair.
- Bootstrap is API-driven (`infra/zitadel/bootstrap.sh`) using a PAT that ZITADEL writes to a shared volume on first init. No realm-import + user-profile-schema post-import dance.
- `Keycloak.AuthServices.*` packages and `Testcontainers.Keycloak` removed from the central catalog (they were forward-pinned but unused).
- `global.json` SDK pin relaxed from `10.0.203` → `10.0.106` with `latestFeature` `rollForward`, so locally-installed SDKs $\geq 10.0.106$ satisfy the build without per-commit `global.json` swapping.
- Supersedes decision #4.

11. Infisical removed; env-var secrets model — 2026-05-05

Context. A custom `InfisicalSecretsConfigurationProvider` had been wired into `Pam.Api` along with a self-hosted Infisical stack (`infisical` , `infisical-postgres` , `infisical-redis`) in compose. Bootstrap (admin, org, project, machine identity) was UI-only and never automated; dev encryption keys lived hardcoded in compose.

Decision. Remove Infisical end-to-end. Production secrets arrive as env vars from whatever orchestrator runs the API; ASP.NET's default configuration precedence (env vars > `appsettings.{env}.json` > `appsettings.json` , with `___` mapping to `:`) handles the rest.

Consequences.

- One fewer service to operate locally. Frees host ports `5434` , `6381` , `8085` .
- No central rotation, audit, or fine-grained access control on secrets. Acceptable until the first real production deploy.
- A dedicated secret store (Vault / SOPS / k3s External Secrets) is now an open ROADMAP entry rather than a half-wired prototype.

10. xUnit v3 for new test code — 2026-05-05

Context. No tests existed yet; we were free to pick the version. The xUnit 2 → 3 migration is real but the cost is bounded by the size of the test surface. Catching it before the test surface is non-zero is much cheaper than catching it later.

Decision. Standardize new test code on `xunit.v3 3.2.2` plus `xunit.runner.visualstudio 3.1.5` (must always match the v3 line — never split).

Consequences.

- One test project (`tests/Pam.Players.UnitTests/`) on xUnit v3 from day one. 21 tests, ~40 ms run.
 - Future test projects copy the same csproj template; no v2-vs-v3 compatibility headaches.
-

9. Persist enum-typed columns as varchar, not int — 2026-05-05

Context. `PlayerStatus` was originally stored as the integer ordinal. Inserting a state mid-list reorders existing values and silently corrupts every existing row. The same trap exists for `ActorType`.

Decision. Persist enums as their name via `HasConversion<string>()` with an explicit `HasMaxLength`. Migrations that change `int` → `varchar` must include an explicit `ALTER ... USING CASE` clause because PostgreSQL will not auto-cast.

Consequences.

- Adding a new enum value is append-only at any position.
 - Column reads as values that mean something to anyone running a `SELECT`.
 - Slight storage overhead vs `int` — negligible.
-

8. Typed `Actor` for `IUserContext` and audit columns — 2026-05-05

Context. Audit columns previously held a single `CreatedBy: string?` that interleaved Keycloak `sub s` (player actions), future operators-realm `sub s` (different realm, but same shape), and the literal string `"system"`. There was no discriminator — a regulator asking "did a Player self-suspend or did an Operator suspend them?" required parsing.

Decision. Replace the string with a typed `Actor(ActorType, Id)` — `ActorType` $\in$ `{System, Player, Operator, Service, Anonymous}`. Persist as a 4-column split: `(CreatedByType, CreatedById, LastModifiedByType, LastModifiedById)`, each `Type` column as `varchar(16)` via `HasConversion<string>()`.

Consequences.

- Audit queries can group/filter by actor type without parsing.
 - Adds two columns per entity. Trivial.
 - Forces every `IUserContext` implementation to map identity into the `Actor` shape — clean migration story when the operators realm lands.
-

7. Domain events dispatch post-save with bounded loop — 2026-05-05

Context. The `DispatchDomainEventsInterceptor` originally hooked `SavingChangesAsync` (pre-save). Any handler with side effects could leak state when the `SaveChanges` that triggered it then failed. The dispatch loop was unbounded `while(true)` — a handler that mutates a tracked aggregate would re-raise events forever.

Decision. Override `SavedChangesAsync` (post-save). Cap re-dispatch at 8 generations and throw on exceed. Drop the sync `SavingChanges` override entirely (forces async-only saves repo-wide).

Consequences.

- Handlers see committed state. No leaked side effects on rollback.
- Lost atomicity-with-DB-write for in-process handlers — acceptable because the only handler today logs. The outbox fills the gap when cross-module broker publishing arrives.
- Bounded-loop throw makes runaway-handler bugs loud instead of hangs.

6. Authorization fallback policy + per-IP rate-limit partitioning — 2026-05-05

Context. No fallback authorization policy meant any new endpoint defaulted to anonymous unless it explicitly called `RequireAuthorization`. The `auth-sensitive` rate-limit policy was a single fixed-window limiter, not partitioned — five attempts a minute total fleet-wide, locking everyone out after the first five.

Decision. Set `AuthorizationOptions.FallbackPolicy` to `RequireAuthenticatedUser` (scheme `players`). Public endpoints opt out via `.AllowAnonymous()`. Convert `auth-sensitive` to a `PartitionedRateLimiter` keyed by `player_id` claim (when authenticated) or forwarded client IP. Configure `ForwardedHeadersOptions` early in the pipeline so the IP key reflects the real client behind a proxy.

Consequences.

- The next endpoint that forgets `RequireAuthorization` gets a 401, not a silent leak.
- Rate-limit windows are per-client, not per-fleet.
- Health and dev OpenAPI/Scalar endpoints must explicitly `.AllowAnonymous()` (already done).

5. Dependency licensing pins — 2026-05-05

Context. Three packages on the catalog were on the edge of going commercial or had already gone commercial in their latest version.

Decision. Stay on the last free release of each:

Package	Version	License pin
MediatR	12.4.1	last MIT (v13+ Lucky Penny commercial)
MassTransit (and rabbitmq / efcore)	8.5.9	last Apache-2.0 of v8 line (v9 Massient commercial)
FluentAssertions	7.2.0	last Apache-2.0 (v8+ Xceed commercial)

`SecurityCodeScan.VS2019` was abandoned (last release 2022-09); removed entirely.

`Microsoft.CodeAnalysis.NetAnalyzers` covers the gap on by default with `AnalysisMode=All`; layered SAST (CodeQL / Sonar security rules) goes in CI when the time comes.

Consequences.

- Zero ongoing license cost. Security patches for v8 MassTransit run through end of 2026; longer-term migration is a 12–18-month decision.
 - Bumping any of these majors is a deliberate license review, not a routine `dotnet outdated` upgrade.
-

4. Stay on Keycloak now; ZITADEL spike at operators-realm milestone — 2026-05-05

Context. Researched Keycloak vs ZITADEL vs Authentik vs ASP.NET Identity (the approach used in the peer `pro-cr-billing` project). PAM is regulated B2C iGaming with self-exclusion, MFA-per-realm, federation, and auditor-checkable password policies — all of which favor an external IDP over in-app Identity.

Decision. Stay on Keycloak for the player-only phase. Schedule a 2-day ZITADEL spike for the moment we're about to add the operators realm — ZITADEL's Org/Project model is a better fit for the planned `Partner` aggregate than two Keycloak realms. ASP.NET Identity ruled out for the player auth path.

Consequences.

- The `IIdentityProvider` interface stays the swap seam; we keep Keycloak-specific concepts (realm name, custom-attribute API shape) on the Keycloak side of it.
 - We accept Keycloak's operational weight (JVM, painful upgrades, realm-import friction) for the duration of the player-only phase.
 - ROADMAP entry "ZITADEL spike (alternative to Keycloak)" tracks this.
-

3. Patterns to lift from `pro-cr-billing` — 2026-05-05

Context. Reviewed the peer billing project's auth and infrastructure code. The auth approach (ASP.NET Identity + custom JWT + RefreshToken rotation) is wrong for PAM but several adjacent patterns are well-shaped and reusable.

Decision. Adopt these regardless of IDP choice:

- `ApiKeyAuthenticationHandler` skeleton — for the future `Pam.GameWallet` host (HMAC-signed game-provider callbacks) and webhooks. **Pending.**
- `ForwardedHeadersOptions` configured with `KnownIPNetworks.Clear()` + `KnownProxies.Clear()`. **Done** (commits `41f3a50`, `7253a51`).
- Rate-limit `OnRejected` with structured logging and a `Retry-After` header. **Done** (`41f3a50`).
- API versioning via `Asp.Versioning.Http` (URL/query/header readers) before more endpoints land. **Pending** (package pinned, not wired).
- `existence` queries on the registration flow (`/exists/email`, `/exists/identification`) — better UX than slow-fail-on-submit. **Pending.**
- Hierarchical role-policy pattern (`Accountant` policy accepts `Owner` too) for `operator.support` $\subset$ `operator.compliance` $\subset$ `operator.admin` when the operators realm lands. **Pending.**

Not lifted: `User : IdentityUser<Guid>`, HS256 with a symmetric key, hardcoded password policy in `AddIdentityCore`, `Database.Migrate()` on startup. Those work for billing's B2B model but regress PAM's regulated B2C posture.

Consequences. PAM's auth direction stays IDP-driven; we get production-quality patterns for the surrounding pipeline without coupling the regulated parts of the system to billing's identity model.

2. Modular monolith with explicit module seams (recap from existing docs) — pre-session

Context, Decision, Consequences. See `ARCHITECTURE.md` sections *Modular monolith, not microservices (yet)* and *Plural module assemblies* — those decisions predate this log and remain authoritative.

1. Keycloak owns credentials; PAM owns the canonical Player (recap from existing docs) — pre-session

Context, Decision, Consequences. See `ARCHITECTURE.md` section *Keycloak owns credentials; PAM owns the canonical player* — predates this log and remains authoritative.

Roadmap

What's deferred and the trigger that should bring each forward.

Up next — ordered punch-list

Phase 3 (`Pam.Identity`) is complete across three PRs. Next is `Players` returning as module #3.

1. **Re-introduce `Pam.Players`** (module #3). Player aggregate scoped under a `Brand` , authenticated via the embedded IDP under a separate audience (`pam_player_api`). Self-service registration + KYC + sessions + limits land as sub-aggregates of the same module.
2. **Test gate before module #4** — outbox + integration tests (Testcontainers: real Postgres) + arch tests (`NetArchTest.Rules`). See [Outbox](#) and [Tests](#).
3. **Module #4 — `Pam.Wallet`** (double-entry ledger). Outbox is non-negotiable from day one of that module. See [the build order](#) below.
4. **`Pam.Notifications` expansion** — module skeleton + `ISender` are in place (`Pam.Notifications` + `Pam.Notifications.Contracts`). Still to come: integration-event consumers (the `Consumers/` folder is empty, waiting for `Players` to publish), templating + locale resolution, send-audit-log `DbContext`, SMS + push transports. Each piece lands when the first consumer needs it. See *Notifications and cross-module email* in `ARCHITECTURE.md` for the pattern.

Deferred — pinned, will land when triggered

`Pam.Identity` (OpenIddict + ASP.NET Core Identity)

Embedded auth module. `OpenIddict AddServer` configured for Authorization Code + PKCE + Refresh (Client Credentials wired but not yet granted to any client). ASP.NET Core Identity for `BackOfficeUser` , roles, permissions. Single `IdentityDbContext` (schema `identity`) hosting Identity tables, `OpenIddict` tables, and the custom `permissions / role_permissions` join. `AddValidation().UseLocalServer()` in the same host — module endpoints validate tokens in-process, no introspection hop.

Phase 3 ships across three PRs.

PR 1 — OIDC plumbing + login + bootstrap. SHIPPED.

- `POST /v1/identity/login` (cookie-based, JSON, anonymous, rate-limited 5/min) — `SignInManager.PasswordSignInAsync` with lockout-on-failure.
- `GET/POST /connect/authorize` — Authorization Code + PKCE, `ConsentTypes.Implicit` for the back-office SPA (no consent UI).
- `POST /connect/token` — code exchange + refresh-token grant, with the claim set refreshed from the live user on every issuance.
- `GET/POST /connect/userinfo` — standard OIDC userinfo gated on scope.
- `GET/POST /connect/logout` — clears the Identity cookie + `OpenIddict` end-session (redirect to `post_logout_redirect_uri`).
- `POST /connect/par` — Pushed Authorization Requests handled natively.
- Token issuance projects `sub` , `email` , `name` , `preferred_username` , `role[]` , `pam_permission[]` , `brand_id` (when set). Destinations split between access and identity tokens by scope; `pam_permission / brand_id` are access-token-only; security stamp never leaves the cookie.

- Idempotent seeder: permissions (from `PermissionCodes.All`), roles (Owner / Manager / Operator / Accountant), role-permission grants (`PermissionCodes.RoleDefaults.*`), `pam_api` scope, the `pam-bo` SPA application descriptor (public client, PKCE required, redirect URIs from `Identity:BackOfficeSpa` config), and the bootstrap Owner from `PAM_BOOTSTRAP_OWNER_EMAIL` / `PAM_BOOTSTRAP_OWNER_PASSWORD` env vars (only seeded if no Owner exists). Wrapped in a Postgres `pg_advisory_xact_lock` (key 100_001) so concurrent replica boots serialise through the seed and can't race UNIQUE constraints.
- Cookie-challenge redirect to the React SPA's `LoginUrl?returnUrl=...` for browser navigations; JSON / `/v1/*` requests get 401 instead.
- Quartz hosted service: hourly cleanup of orphaned tokens and authorizations.
- `Pam.Api` fallback authorization policy requires bearer auth via the OpenIddict validation scheme; one `Permissions.<code>` policy per permission code, with `platform.admin` granting every other policy via assertion.

PR 2 — back-office user management + MFA. SHIPPED.

All endpoints under `/v1/identity/...`, gated on the matching permission.

- `POST /v1/identity/users` — admin creates a back-office user. Back-office users are never self-service; only Owner/Manager can add them. Permission: `identity.users.write`.
- `GET /v1/identity/users` (paged: `page`, `pageSize`, optional `brandId` / `role` / `lockedOut`) and `GET /v1/identity/users/{id}`. Permission: `identity.users.read`.
- `PATCH /v1/identity/users/{id}` — email, `brandId`, `lockoutEnabled`. Status flags (`EmailConfirmed`, `TwoFactorEnabled`) mutate through their own endpoints because they carry side effects.
- `DELETE /v1/identity/users/{id}` — soft-delete via `LockoutEnd = DateTimeOffset.MaxValue` + security-stamp rotation. Hard-delete violates regulatory retention.
- `POST /v1/identity/users/{id}/roles` / `DELETE /v1/identity/users/{id}/roles/{role}` — idempotent. Each change rotates the security stamp so token claim sets refresh. Permission: `identity.roles.write`.
- `POST /v1/identity/users/{id}/unlock` — clears the lockout end + resets failed-access count; refuses on soft-deleted users.
- `POST /v1/identity/me/change-password` — authenticated user changes their own password (knows the current one). Rate-limited.
- `POST /v1/identity/me/mfa/enroll` returns the base32 shared key + an `otpauth://` URI for QR rendering. `POST /v1/identity/me/mfa/verify` verifies the TOTP and flips `TwoFactorEnabled`.
- `POST /v1/identity/login/mfa` — MFA challenge step using `SignInManager.TwoFactorAuthenticatorSignInAsync`.
- `CreateBrand` and `GetBrand` now require `Permissions.operators.brands.write` and `Permissions.operators.brands.read` respectively (no more `.AllowAnonymous()`).
- `HttpContext` replaces the default `SystemUserContext` so audit columns reflect the authenticated operator's OIDC sub (with fallback to `Actor.System` for startup work / `Actor.Anonymous` for anonymous endpoints).
- Tests: validator suites for `CreateUser` / `ListUsers` / `ChangePassword` / `MfaVerify`, plus `PermissionResolver` against the EF Core in-memory provider. 33 Identity tests + 18 Operators tests, all green.

PR 3 — email flows + MFA durability + revocation tightening. SHIPPED.

Bundled three things: the ROADMAP-planned email flows, plus two pieces that proved necessary once MFA was actually in use (recovery + admin reset), plus the validation-stack hardening that makes revocation feel instant.

- `POST /v1/identity/forgot-password` (anonymous, anti-enumeration: always 204) — issues `userManager.GeneratePasswordResetTokenAsync` and sends the reset link via `ISender` only when the email exists and is confirmed.
- `POST /v1/identity/reset-password { email, token, newPassword }` — consumes the token, sets the new password. Rotates the security stamp so sessions on other devices die.
- `POST /v1/identity/confirm-email { email, token }` — anonymous; user clicks the link from email and posts back.
- `POST /v1/identity/users/{id}/send-confirmation-email` — admin re-send (gated on `identity.users.write`). Auto-fires on `POST /v1/identity/users` creation; SMTP failure is logged, not fatal — admin can retry.
- `POST /v1/identity/me/mfa/recovery-codes` — generates 10 one-time codes via `GenerateNewTwoFactorRecoveryCodesAsync`. Plaintext returned once; previously-issued codes get invalidated on regenerate.
- `POST /v1/identity/login/recovery-code { code }` — completes the partial sign-in using a recovery code instead of a TOTP (one-time use).
- `POST /v1/identity/me/mfa/disable { currentPassword }` — self-disable with password challenge; clears the authenticator key so re-enable starts fresh.
- `POST /v1/identity/users/{id}/mfa/reset` — admin reset (gated on `identity.users.write`). Clears authenticator, disables 2FA, rotates security stamp.
- Email sender: `ISender` + `SmtpSender` (MailKit) targeting the new `pam-mailpit` dev container at `localhost:1025`. `Identity:Smtp` config block carries host/port/credentials/from-address — production swap is a config change.
- OpenIddict validation now uses `EnableAuthorizationEntryValidation()` + `EnableTokenEntryValidation()` — revocation propagates immediately instead of waiting for the security-stamp validation window. `/connect/revocation` and `/connect/introspect` URIs registered on the server; the back-office SPA descriptor gets revocation+introspection endpoint permissions.

Deferred indefinitely

- Self-service back-office registration. Wrong threat model — anyone with `POST /v1/identity/users` is a privilege-boundary breach. The bootstrap Owner is the on-ramp.
- Dynamic client registration (`/connect/register`). We own all the clients; only `pam-bo` exists today, future module-to-module clients are seeded.

Player auth — Phase 4, not Phase 3.

`POST /v1/auth/register` (public, self-service) lands when the Players module returns. Different scope set, different audience under the same OpenIddict server.

Reference patterns: `pro-cr-billing`'s `User : IdentityUser<Guid>` shape (lockout, password policy) carried over; `pro-cr-billing`'s custom JWT + RefreshToken rotation NOT carried over (OpenIddict's token storage replaces it).

Outbox + transactional integration-event publishing — SHIPPED

MassTransit EF Core outbox is wired on `OperatorsDbContext`. Integration events published from inside a `SaveChanges` scope (`BrandCreatedDomainHandler` and equivalents) write to `outbox_message` in the same transaction as the aggregate write; an `EntityFrameworkOutboxDeliveryService` forwards to RabbitMQ.

`DispatchDomainEventsInterceptor` moved from post-save to pre-save so the dispatch happens inside the transaction. See *Outbox + pre-save domain-event dispatch* in ARCHITECTURE.md for the new semantics and

their trade-offs.

Per-module wiring: each publishing module exposes a `ConfigureOutbox(IBusRegistrationConfigurator)` delegate, composed in `Program.cs`'s call to `AddPamMassTransit`. Adding a new publisher takes three steps (package, EF model entities, `ConfigureOutbox`).

For Wallet, this stays non-negotiable on day one of that module.

Audit module

- **What:** `Pam.Audits` module, append-only, subscribes to all integration events. Records actor, IP, correlation-id, raw event payload with hash chaining.
- **Trigger:** regulator asks, or first jurisdiction we operate in requires it (most do).
- **Note:** this is **regulatory** audit. Application audit (the `created_by_type` / `created_by_id` / `last_modified_*` columns on every `Entity<TId>`) already covers app-level "who did what."

Brand-scoped global query filters

- **What:** each `DbContext` adds an EF Core global query filter on `BrandId` so cross-brand reads are impossible by default once back-office endpoints arrive. Operator endpoints can elevate to cross-brand access via a specific permission (`operator.platform-admin`).
- **Trigger:** when the second brand-scoped aggregate (Player) lands. Adding the filter retroactively across N modules is N migrations and N code reviews; doing it once at module-#3-start is one PR.

Shared signing material + DP keys — SHIPPED

Two pieces that have to be in place before a second replica can serve traffic alongside the first:

- **Data Protection master keyring** persists to `identity.data_protection_keys` via `IDataProtectionKeyContext` on `IdentityDbContext`. Cookies + `OpenIdDict` state strings issued by one replica validate on another. `SetApplicationName(...)` isolates keys across environments sharing a database (`pam-api/Development` $\neq$ `pam-api/Production`).
- **OpenIdDict signing + encryption certificates** load from PFX files configured under `OpenIdDict`: `{Signing,Encryption}Certificate:{Path,Password}` in non-Development. Mount the same PFX into every replica; rotate by swapping the file and rolling restarts. Development keeps the persisted self-signed certs from `AddDevelopment{Signing,Encryption}Certificate`.

Without either of these, tokens and cookies become per-instance state and $N > 1$ replicas can't share user sessions.

Distributed rate limiter — SHIPPED

The `auth-sensitive` and `api-default` policies now use the `RedisRateLimiting` package (cristipufu) instead of the in-memory limiters. A single `IConnectionMultiplexer` registered as a DI singleton backs both — `RedisFixedWindowRateLimiter` for `auth-sensitive`, `RedisSlidingWindowRateLimiter` for `api-default`.

Failure mode: `AbortOnConnectFail=false` so a brief Redis outage doesn't prevent boot, but rate-limited endpoints surface 5xx while Redis is unreachable. Intentional — fail-open on `/login` is worse than a brief outage. Health check (`identity-db`, `redis`) catches it.

Sliding-window semantics changed slightly: the Redis impl uses a sorted set keyed by request timestamp (textbook sliding window), not segmented buckets, so `SegmentsPerWindow` is gone. More accurate, not a regression.

OTLP exporter + collector — SHIPPED

Traces, metrics and logs now leave the API over OTLP. Local dev uses the `grafana/otel-lgtm` all-in-one container (Tempo + Mimir + Loki + Grafana)

- Collector — Grafana UI on `:3001`, OTLP gRPC on `:4317`, HTTP on `:4318`). Resource attributes (`service.name`, `service.version`, `deployment.environment`, `host.name`) are emitted on every signal.

Instrumentation sources: `AspNetCore` + `HttpClient` + `EF Core` + `Npgsql` + `MassTransit` + `Pam.*`

ActivitySources/Meters. Logs flow through Serilog's OpenTelemetry sink alongside the existing Console/Seq sinks.

Production swaps the endpoint via `OTEL_EXPORTER_OTLP_ENDPOINT` — Grafana Cloud, self-hosted LGTM split out, Honeycomb, Datadog, ...

CorrelationId middleware — SHIPPED

`CorrelationIdMiddleware` in `Pam.Shared.Http` reads `X-Correlation-Id` (generates a `Guid.CreateVersion7().ToString("N")` if absent), echoes it on the response, pushes it into the current Activity's baggage + tag (so `MassTransit`'s diagnostic instrumentation carries it through bus publishes and the outbox), and `BeginScope` s a `CorrelationId` log property so every log line in the request is correlated.

Wired in `Program.cs` immediately after `UseForwardedHeaders` and before `UseSerilogRequestLogging` so the request-log entry itself carries the id.

Architecture tests — SHIPPED

`tests/Pam.ArchitectureTests` (`xUnit` + `NetArchTest.Rules`) encodes the module-boundary and DDD-shape rules from `ARCHITECTURE.md`:

- **Module boundary:** each `Pam.<X>` assembly must not reference another `Pam.<Y>` directly. `*.Contracts` is the only allowed seam.
- **Domain events:** types ending in `DomainEvent` must implement `IDomainEvent`.
- **Integration events:** types in `*.IntegrationEvents` namespaces must end in `IntegrationEvent`.
- **Aggregates:** concrete types implementing `IAggregate` must derive from `Aggregate<>`.

Fails the test run if a future PR introduces a coupling that breaks the extraction-to-microservice plan. 8 tests; sub-second run.

Production secret store

- **Status:** env-var-driven. Orchestrator (systemd / k3s / Swarm) injects secrets as env vars; ASP.NET maps `___` → `:` over `appsettings.{env}.json`. No in-process secret-fetching code.
- **Open:** pick a dedicated secret store when there's a real production deploy target. Candidates: HashiCorp Vault (most flexible, heaviest ops), SOPS + age (file-based, GitOps-friendly, no server), k3s External Secrets (cleanest if we land on k3s).
- **Trigger:** first staging or production environment that needs more than orchestrator-managed env vars (rotation, audit, fine-grained access controls, multi-team isolation).

Tests

- **Unit tests:** 70 across `Pam.Operators.UnitTests` (Brand aggregate behavior + `CreateBrand` validator) and `Pam.Identity.UnitTests` (validators for `CreateUser` / `ListUsers` / `ChangePassword` / `MfaVerify` / `MfaDisable` / `ForgotPassword` / `ResetPassword` / `ConfirmEmail` / `LoginRecoveryCode` + `PermissionResolver` against the EF Core in-memory provider). xUnit 3 + FluentAssertions 7, ~1s.

- **Architecture tests:** 8 in `tests/Pam.ArchitectureTests` using `NetArchTest.Rules`. Module boundary + DDD shape rules. See *Architecture tests* — *SHIPPED* above.
- **Integration tests:** `tests/Pam.IntegrationTests` boots the API via `WebApplicationFactory<Program>` against Testcontainers (Postgres 17, RabbitMQ 4, Redis 7). `PamContainersFixture` is a shared `ICollectionFixture` so the ~10–20s container boot cost is paid once. Initial coverage: live + ready health probes, anonymous `POST /v1/operators/brands` → 401. 3 tests, ~3s including container warm-up.
- **What's left:** expand integration coverage as new endpoints land (the test gate is the harness, not the coverage). Authenticated flows need a test-only auth seam (issue a dev bearer token) so the full `CreateBrand` → `GetBrand` round-trip can run authenticated.

Legacy `gbs-db` cutover plan

- **What:** a written strangler-fig design covering (a) which endpoints route to new PAM vs legacy `GBS-BAS`; (b) data sync direction during cutover (one-way pull from `gbs-db` into PAM, or dual-write); (c) reconciliation strategy for balances/state that exist in both stores during overlap; (d) rollback story per cutover slice.
- **Trigger:** before any PAM module starts owning reads/writes for data that production currently serves from `gbs-db`. The 100k existing players, 357 tables, and 1,901 stored procs are a real migration project, not an afterthought.

Modules not yet built

In dependency / priority order:

1. **Pam.Identity** (Phase 3, in flight) — back-office Users & Agents, OpenIdDict, roles + permissions.
2. **Pam.Players** — Player aggregate scoped under a Brand. KYC, sessions, limits land as sub-aggregates of this module to start; spin out as separate modules when the surface justifies it.
3. **Pam.Wallet** — double-entry ledger, deposits, withdrawals, holds, balance projections. Outbox required from day one. Most regulated module.
4. **Pam.Limits** (if not already inside Players) — deposit / loss / session limits, cooling-off, self-exclusion. Hooks into the MediatR pipeline as a `LimitsBehavior` so any `IPlayerInitiated` command is checked before the handler.
5. **Pam.Bonuses** — grants, wagering progress, expiry. Subscribes to wallet events. Bonus engine via a typed expression DSL (see `CORE_PLATFORM_MAPPING.md` decision #3).
6. **Pam.Bets** — internet bets + bet shops; Wallet + GameCatalog dependencies.
7. **Pam.GameProviders** / **Pam.GameCatalog** — provider integrations, product catalog.
8. **Pam.Affiliates** — own admin panel, commissions, referral tracking.
9. **Pam.Notifications** — email/SMS/push. Subscribes to Player/KYC/Wallet events.
10. **Pam.Reporting** — read-side projections over events; can be incrementally added.
11. **Pam.Audits** — append-only, subscribes to everything (see above).

Game wallet ingress (`Pam.GameWallet`) is a **separate host**, not a module. Game providers hit it with HMAC-signed `/auth`, `/bet`, `/win`, `/rollback` calls; latency budget is sub-200ms p99. It calls into `Pam.Wallet` via the contracts interface but does not share the back-office API request pipeline.

Smoke-test caveats

Redis + Seq compose port conflicts

`docker-compose.yml` declares both, but they default to standard ports (6379 , 5341) which conflict with the user's `dmt-redis` and `dmt-seq` peer-project containers. Decide whether to renumber ours or stop the others when the API actually needs them (Redis: distributed rate limiting; Seq: log viewing).

`mprocs.log` and `dotnet watch`

If `mprocs` writes its log into the project tree, `dotnet watch` may treat that log as a file change and trigger a hot reload loop. Exclude the log file from watch via `<Watch Remove=".../mprocs.log" />` in `Directory.Build.props` or the `Pam.Api` csproj when this becomes annoying.

Local development

Prerequisites

- .NET 10 SDK (10.0.106 or later patch — pinned in `global.json` with `latestFeature` `rollForward`)
- Docker (or Podman) with Compose
- `make`

First-time setup (mprocs — recommended)

```
mprocs                                # one command; two panes
```

`mprocs.yaml` runs two procs:

- `services` — `docker compose up` (Postgres, RabbitMQ, Redis, Seq, Grafana LGTM) in foreground
- `api` — `make dev-api` (apply migrations + `dotnet watch`)

Switch panes with `Tab` ; quit with `q` or `Ctrl-C` .

First-time setup (manual)

```
make up                                # docker compose up -d
make migrate-update MODULE=Operators  # apply EF migrations
make dev-api                          # dotnet watch
make test                             # unit tests in a separate terminal
```

URL	What
<code>http://localhost:5000</code>	PAM API
<code>http://localhost:5000/scalar/v1</code>	Scalar OpenAPI UI
<code>http://localhost:8090</code>	Seq logs (no auth in dev)
<code>http://localhost:3001</code>	Grafana — traces (Tempo), metrics (Mimir), logs (Loki)

Smoke test

```
# Create a brand.
curl -i -X POST http://localhost:5000/v1/operators/brands \
  -H "Content-Type: application/json" \
  -d '{"name": "BetAnything EU", "slug": "betanything-eu", "jurisdiction": "EU"}'
# → 201 Created
#   Location: /v1/operators/brands/<brand-id>
#   { "id": "<brand-id>" }

# Read it back.
curl -s http://localhost:5000/v1/operators/brands/<brand-id> | jq
```

The API is anonymous in Phase 2. Authentication arrives in Phase 3 with the `Pam.Identity` module (OpenIddict + ASP.NET Identity); endpoints will then default to `RequireAuthorization` and individual ones opt out via `.AllowAnonymous()`.

Service ports

Service	Host port	Notes
Postgres	5432	user: <code>pam</code> , db: <code>pam</code>
RabbitMQ	5672 (amqp), 15672 (UI)	user: <code>pam</code>
Redis	6379	password: <code>redis_dev_password</code>
Seq	5341 (ingest), 8090 (UI)	optional log viewer
otel-lgtm	4317 (OTLP gRPC), 4318 (OTLP HTTP), 3001 (Grafana UI)	traces+metrics+logs (Tempo/Mimir/Loki)
Pam.Api	5000	

Known port conflicts

If you have a peer project (`dmt-redis` , `dmt-seq`) running with the same defaults, our `redis` and `seq` services will fail to start. Either:

- Stop the conflicting containers (`docker stop dmt-redis dmt-seq`), or
- Skip those services for now — neither is required to run the API. Logs go to console; Redis only matters once distributed rate limiting goes in.

EF migrations

The Makefile is parameterized by `MODULE` :

```
make migrate-add MODULE=Operators NAME=AddSomething
make migrate-update MODULE=Operators
make migrate-status MODULE=Operators
make migrate-remove MODULE=Operators      # removes the last unapplied migration
```

A `IDesignTimeDbContextFactory<OperatorsDbContext>` lives in `src/Modules/Operators/Pam.Operators/Data/` , so EF tooling can spin up the context standalone. The connection string can be overridden via `PAM_DESIGN_CONNECTION` env var (defaults to the local compose).

`Microsoft.EntityFrameworkCore.Design` is a `PackageReference` on `Pam.Api` (the startup project). EF Tools require it there even though the design-time factory is in the module project — the error message ("Your startup project doesn't reference ...") is misleading.

Adding a feature

The `CreateBrand` feature in `src/Modules/Operators/Pam.Operators/Brands/Features/CreateBrand/` is the canonical template. Four files per feature:

```

<FeatureName>/
├─ <FeatureName>Command.cs      # ICommand<TResponse> or IQuery<TResponse>
├─ <FeatureName>Validator.cs    # AbstractValidator<TCommand>
├─ <FeatureName>Handler.cs      # ICommandHandler<,> or IQueryHandler<,>
└─ <FeatureName>Endpoint.cs     # ICarterModule with one MapPost/Get/...

```

MediatR + FluentValidation auto-discover via assembly scanning. Carter discovers endpoints via the `DependencyContextAssemblyCatalog` registered in `Program.cs`. No manual DI registration needed.

Domain events live alongside the aggregate (`<Concept>/Events/<Event>.cs`); a sibling `<Concept>/EventHandlers/<Event>DomainHandler.cs` listens for the domain event and publishes the corresponding integration event from `Pam.<Module>.Contracts`.

Adding a module

Mirror `Pam.Operators`. Three projects per module, plus the test project:

```

src/Modules/<Module>/
  Pam.<Module>/                # csproj + <Module>Module.cs at root
  Pam.<Module>.Contracts/      # DTOs, IQuery<T>, integration events
tests/Pam.<Module>.UnitTests/

```

`<Module>Module.cs` exposes two extension methods used by `Pam.Api`:

- `AddXModule(this IServiceCollection, IConfiguration)` — registers `DbContext`, interceptors, health checks.
- `UseXModuleAsync(this IServiceProvider)` — runs `MigrateAsync` on startup.

Interceptors (`AuditableSaveChangesInterceptor`, `DispatchDomainEventsInterceptor` from `Pam.Shared`) are registered with `services.TryAddScoped<...>()` so multiple modules don't conflict.

Persistence uses schema-per-module (`modelBuilder.HasDefaultSchema("...")`) plus `snake_case` columns (`options.UseSnakeCaseNamingConvention()` from the `EFCore.NamingConventions` package — apply on **both** runtime and design-time `DbContext` setup). Health probe via `AddHealthChecks().AddNpgsql(connectionString, name: "<module>-db", tags: ["ready", "module:<module>"])`.

After scaffolding the module, register it in `Pam.Api`:

```

var moduleAssemblies = new[] { typeof(OperatorsModule).Assembly /*, ...*/ };

builder.Services.AddPamMediatR(moduleAssemblies);
builder.Services.AddCarter(new DependencyContextAssemblyCatalog(moduleAssemblies));
builder.Services.AddOperatorsModule(builder.Configuration);
// later: await app.Services.UseOperatorsModuleAsync();

```

Where things live

Concern	Location
Cross-cutting infra (DDD primitives, behaviors, exceptions, interceptors)	<code>src/Shared/Pam.Shared/</code>
CQRS interfaces, IDomainEvent, Actor, PamIds	<code>src/Shared/Pam.Shared.Contracts/</code>
MassTransit registration, IntegrationEvent base	<code>src/Shared/Pam.Shared.Messaging/</code>
Per-module aggregates, features, EF, wire-up	<code>src/Modules/<Module>/Pam.<Module>/</code>
Public DTOs, queries, integration events	<code>src/Modules/<Module>/Pam.<Module>.Contracts/</code>
API host, DI wiring, OTel, health, OpenAPI	<code>src/Bootstrapper/Pam.Api/</code>

Secrets

Local dev: `appsettings.json` carries non-secret defaults.

Production: secrets arrive as env vars from whatever orchestrator runs the API (systemd unit, k3s Secret, Swarm secret). ASP.NET's default configuration precedence reads env vars over `appsettings.{env}.json` and treats `__` as the nesting separator. A dedicated secret store (HashiCorp Vault, SOPS, k3s External Secrets, etc.) is open — see [ROADMAP.md](#).

Observability (OpenTelemetry)

Pam.Api emits traces, metrics and logs over OTLP to the local Grafana LGTM container (`otel-lgtm` in compose). Open Grafana at `http://localhost:3001` — Tempo (traces), Mimir (metrics) and Loki (logs) are pre-wired as datasources. Resource attributes (`service.name` , `service.version` , `deployment.environment` , `host.name`) tag every signal so you can segment by env/instance once more than one runs.

If LGTM isn't running, the SDK retries OTLP exports in the background and logs warnings — the API itself keeps serving. To point at a different collector (Grafana Cloud, a real Tempo, Honeycomb, Datadog, ...) set `OTEL_EXPORTER_OTLP_ENDPOINT` (and for the Serilog log sink, `Serilog:WriteTo:2:Args:Endpoint` via env var) at the orchestrator.

Scalar / OpenAPI

Scalar UI is dev-only (`if (app.Environment.IsDevelopment())`). To enable in staging or behind admin auth, pull `MapOpenApi()` and `MapScalarApiReference()` out of the dev-only block in `Program.cs` and add the appropriate `RequireAuthorization` chain.

Common gotchas

- `dotnet run --no-build` runs the cached DLL — newly generated migrations won't be picked up. Always `dotnet build` after `make migrate-add` before re-running the API.
- `BannedSymbols.txt` blocks `DateTime.UtcNow` , `DateTime.Now` , and `DateTimeOffset.Now` . `DateTimeOffset.UtcNow` is allowed (intentional — see the `IntegrationEvent` base record).

- `dotnet watch` watches files reachable from `Pam.Api`. If `mprocs.log` (or any other runtime artifact) ends up triggering hot reload, exclude it via `<Watch Remove=".." />` in the relevant csproj or a `Directory.Build.props` entry.

Platform hardening — 2026-05-11

This document captures the platform-hardening pass: what we did, why, and what's now possible that wasn't before. It complements [ARCHITECTURE.md](#) (the *why* behind the design) and [ROADMAP.md](#) (the trigger-gated punch list).

The prompt

After Phase 3 (`Pam.Identity`) shipped, a technical assessment scored the system **38 / 100** against the question "can this handle millions of records / requests today?" The score was low because:

- Two of the three modules that actually hold the data — `Pam.Players` , `Pam.Wallet` — didn't exist.
- The system had never proven it could run with more than one replica: in-memory rate limiter, ephemeral Data Protection keys, ephemeral OpenIddict signing material, race-prone startup seeder.
- No outbox: integration events could leak or vanish under crash.
- No integration tests, no architecture tests, no end-to-end harness.
- No telemetry leaving the process.

The work below took that 38 to ~62–65 by closing the *pre-replica* blockers, establishing the test gate, and scaffolding the two remaining modules that will host the bulk of the data.

What landed (seven PRs, in dependency order)

1. OTLP pipeline + Grafana LGTM — `chore/otel-pipeline`

Traces, metrics, and logs leave `Pam.Api` over OTLP. Local dev runs the `grafana/otel-lgtm` all-in-one container (Tempo + Mimir + Loki + Grafana

- Collector — Grafana UI on `:3001` , OTLP gRPC on `:4317` , HTTP on `:4318`). Production swaps the endpoint via `OTEL_EXPORTER_OTLP_ENDPOINT` without code changes.

Resource attributes (`service.name` , `service.version` , `deployment.environment` , `host.name`) tag every signal so dashboards can segment by env/instance once more than one runs. Instrumentation: `AspNetCore`, `HttpClient`, `EF Core`, `Npgsql`, `MassTransit`, `Pam.*` `ActivitySources/Meters`. Logs flow through Serilog's OpenTelemetry sink alongside the existing Console/Seq sinks.

2. Shared signing material + DP keys — `chore/multi-replica-keys`

Two pieces of state that have to be shared before a second replica can serve traffic alongside the first:

- **Data Protection master keyring** persists to `identity.data_protection_keys` via `IDataProtectionKeyContext` on `IdentityDbContext`. Cookies + OpenIddict state strings issued by one replica now validate on another. `SetApplicationName(...)` isolates keys across environments sharing a database (`pam-api/Development` $\neq$ `pam-api/Production`) — keys from staging won't decrypt prod cookies, which is the desired behavior.
- **OpenIddict signing + encryption certificates** load from PFX paths configured under `OpenIddict: {Signing,Encryption}Certificate: {Path,Password}` in non-Development. Mount the same PFX into every replica; rotate by swapping the file and rolling restarts. Development keeps the persisted self-signed certs from `AddDevelopment*Certificate`.

Why Postgres for DP keys, not Redis. Security material shouldn't be tied to cache availability — losing Redis (cache) shouldn't invalidate every cookie. Identity already has Postgres, so no new dependency.

Why PFX files, not env-var-base64-PFX. Maps cleanly to k8s Secret-mounted-as-file, systemd `LoadCredential=`, and Swarm secrets.

Fail-fast outside Development if either cert path is unset — silently falling back to ephemeral keys would break auth subtly.

3. Redis-backed rate limiter — `chore/redis-rate-limiter`

The in-memory `FixedWindow` / `SlidingWindow` limiters partition state per-process. Two replicas behind a load balancer each track their own counts, so an attacker hitting `/login` at 2× the configured rate gets served by alternating instances. Both policies now use the `RedisRateLimiting` package's distributed variants (`RedisFixedWindowRateLimiter` for `auth-sensitive`, `RedisSlidingWindowRateLimiter` for `api-default`).

A single `IConnectionMultiplexer` registered as a DI singleton backs the limiter, the new Redis health check, and any future module needing a shared cache. `AbortOnConnectFail=false` avoids blocking boot on a briefly-unreachable Redis; rate-limited endpoints surface 5xx while Redis is down — intentional fail-closed posture for auth.

`SegmentsPerWindow` drops from the sliding-window policy: the Redis impl uses a sorted-set sliding window keyed by timestamp (textbook), not segmented buckets. More accurate, not a regression.

4. EF Core outbox + pre-save dispatch — `chore/operators-outbox`

The publish-then-commit-fail race window: a crash between `IPublishEndpoint.Publish` and the broker's ack drops the event; a failed `SaveChanges` after a successful publish leaves a phantom event referring to a row that never existed. Both produce states consumers and audit can't recover.

Wiring. `MassTransit.EntityFrameworkCore` on `Pam.Operators`; `OperatorsDbContext` adds inbox/outbox model entities. `OperatorsModule.ConfigureOutbox` composes the bus-side outbox setup (`UsePostgres`, `UseBusOutbox`, `QueryDelay`, `DuplicateDetectionWindow`); `Program.cs` passes it through `AddPamMassTransit`'s new `configureBus` parameter. Future publishing modules add their delegate the same way.

Pre-save dispatch. `DispatchDomainEventsInterceptor` moves from `SavedChangesAsync` to `SavingChangesAsync`. This is the precondition for the outbox — handlers' `IPublishEndpoint.Publish` calls now enrol in the same transaction as the `SaveChanges` that triggered them, so `outbox_message` rows commit atomically with the aggregate write. A background `EntityFrameworkOutboxDeliveryService` polls the table and forwards to RabbitMQ.

Trade-offs locked in.

- Domain handlers see pre-commit state. Queries against freshly-written rows return nothing — pass data through the event payload instead.
- A throwing handler rolls back the whole `SaveChanges`. Atomic by design. Don't put best-effort side effects (logging, metrics) in domain handlers; put them in integration-event consumers.

5. Concurrent-boot seeder lock — `chore/seeder-advisory-lock`

The seeder is idempotent per-step but the *steps* race when two replicas boot concurrently: replica A and replica B both notice the Owner role missing, both call `roleManager.CreateAsync`, the second fails with a UNIQUE violation that surfaces as a 500 during startup. EF migrations already serialise through `__EFMigrationsHistory`, but seeding had no such lock.

`IdentitySeeder.SeedAsync` is now wrapped in a transaction + `pg_advisory_xact_lock(100_001)`. The `xact` variant releases on commit/rollback so a crash mid-seed doesn't orphan the lock. Once replica A finishes, replica B acquires, finds every step's precondition met (idempotent), commits quickly.

Reserve **100_000–100_999** for module bootstrap seeders so future modules don't have to think about lock-key collisions per-module.

6. Test gate — `chore/test-gate`

Three pieces ROADMAP gated Wallet on.

Integration tests (`tests/Pam.IntegrationTests/`)

- `PamContainersFixture` boots Postgres 17 + RabbitMQ 4 + Redis 7 via Testcontainers, shared as an `ICollectionFixture` so the ~10–20s boot cost is paid once per test session.
- `PamApiFactory : WebApplicationFactory<Program>` overrides `ConnectionStrings` + `MessageBroker` via `ConfigureAppConfiguration` — services-collection overrides are too late because `Program.cs` reads config **before** `builder.Build()` (the Redis multiplexer is awaited at the top level). RabbitMQ creds come from the container's connection string, not `guest/guest`.
- Initial coverage: live + ready health probes, anonymous create-brand → 401 (auth fence intact). The point of this PR is the harness, not full coverage — coverage grows when endpoints land.

Architecture tests (`tests/Pam.ArchitectureTests/`) — 8 NetArchTest rules:

- Module boundary: `Pam.<X>` may not reference `Pam.<Y>` directly. Only `Pam.<Y>.Contracts` is allowed.
- Domain events ending in `DomainEvent` must implement `IDomainEvent`.
- Types in `*.IntegrationEvents` namespaces must end in `IntegrationEvent`.
- Aggregates implementing `IAggregate` must derive from `Aggregate<>`.

CorrelationId middleware (`src/Shared/Pam.Shared/Http/`). Reads `X-Correlation-Id`, generates `Guid.CreateVersion7().ToString("N")` if absent, echoes on response, sets the id on `Activity.Current` as both baggage and a tag, `BeginScope`'s a `CorrelationId` log property.

Registered before `UseSerilogRequestLogging` so the request-log entry itself carries the id. MassTransit's diagnostic instrumentation propagates Activity baggage through publishes, so the outbox carries the id to async consumers — exactly the gap OTel's `traceparent` doesn't cover.

7. Module #3 + #4 scaffolds — `feat/players-scaffold` + `feat/wallet-scaffold`

Both follow the established per-module pattern (`Pam.<X>` / `Pam.<X>.Contracts` / `tests/Pam.<X>.UnitTests` ; module class with `Add*Module` / `Use*ModuleAsync` ; schema-per-module + `snake_case`; design-time factory; one placeholder test).

Players — schema `players`. `Player(BrandId, Email)` placeholder. Composite UNIQUE (`brand_id, email`) because email uniqueness is per-brand, not global — different brands can legitimately share player emails. `PlayersDbContext.OnModelCreating` leaves a placeholder comment for the brand-scoped global query filter; wiring requires `IBrandContext` reading from the JWT `brand_id` claim, which lands with the first authenticated player endpoint.

Wallet — schema `wallet`. `Account(BrandId, PlayerId, Currency)` placeholder. Composite UNIQUE (`brand_id, player_id, currency`) — one account per (player, currency) per brand. ISO 4217 currency, fixed-length 3. Real double-entry ledger (`LedgerEntry` / `Transaction` with sum-to-zero invariants, balance snapshots, multi-currency conversion) lands in follow-up PRs.

Wallet outbox is wired on day one. `WalletDbContext.OnModelCreating` calls `AddInboxStateEntity` / `AddOutboxMessageEntity` / `AddOutboxStateEntity`; the initial migration provisions the tables; `WalletModule.ConfigureOutbox` is composed with `OperatorsModule.ConfigureOutbox` in `Program.cs`'s call to `AddPamMassTransit`. Ledger writes will commit atomically with their `LedgerEntryPosted` integration event from the very first feature PR — non-negotiable per ARCHITECTURE.md.

Where this leaves the assessment

Capability	Before	After
Run N>1 replicas behind a load balancer	No — cookies/tokens/rate-limit state per-instance	Yes — shared via Postgres + Redis
Integration events survive a crash	No — publish-then-commit-fail loses messages	Yes — outbox commits atomically with the row
Concurrent-replica boot	Race on UNIQUE constraints in seed	Serialised on <code>pg_advisory_xact_lock</code>
End-to-end debugging across async flows	Partial — OTEL covers HTTP ↔ HTTP only	<code>CorrelationId</code> fills the async gap
Telemetry leaves the process	No	OTLP traces + metrics + logs
Module-boundary rules enforced	Code review only	NetArchTest gate
Real-DB integration tests	None	Testcontainers harness + 3 representative tests
Modules holding the data	Operators + Identity + Notifications (skeleton)	+ Players + Wallet (both scaffolded, Wallet outbox-ready)

Assessment score moves from **38** → **~62–65**. The remaining gap to break 70 is real Wallet domain code (double-entry ledger + partitioning), a real Players model (KYC, sessions, limits), and the read-side / cache / CDC story that turns "tables exist" into "millions of requests routed off the write DB."

What's still gating higher scale

In rough order of impact, the next things to attack:

- 1. Real Wallet domain code.** `LedgerEntry` / `Transaction` with sum-to-zero invariants, balance snapshots, multi-currency conversion. This is the module that will actually hold the "millions of records." Partitioning strategy (time-based on `outbox_message` and `ledger_entry`), hot-path indexes, and append-only patterns get designed during this work.
- 2. Brand-scoped global query filter.** Placeholders are in both `PlayersDbContext` and `WalletDbContext`. Needs an `IBrandContext` reading the JWT `brand_id` claim, mirrored from `HttpContext`. Lands with the first authenticated player / wallet endpoint.
- 3. Read-side scaling.** Pagination is offset-based today (`page` , `pageSize`). Cursor pagination on the high-volume reads (Players list, Wallet ledger) will be needed before record counts climb. No read replicas, no projection store, no L2 cache wired yet — Redis is provisioned but only used for rate limiting.
- 4. Auth-aware integration tests.** The current harness verifies the 401 fence. A test-only auth seam (issue a dev bearer token) lets create-brand → get-brand round trip run authenticated.
- 5. Audit module.** Regulatory audit (immutable, hash-chained) is still deferred. First jurisdiction with a hard requirement triggers it.

6. **Production secret store + CI/CD.** Env-var-driven config works for now; HashiCorp Vault / SOPS / k3s External Secrets gets picked when a real production deploy target lands. No CI pipeline exists yet — `make test` is the only gate.

See [ROADMAP.md](#) for the trigger-gated punch list and the SHIPPED entries for each of the items above.

Testing

This document is the topical reference for the test suites: what each suite covers, why we drew the lines where we did, what's intentionally **not** tested yet, and how to run them.

For the narrative behind the test gate landing, see [PLATFORM_HARDENING.md](#); for the trigger-gated "what's next" view, see the *Tests* section of [ROADMAP.md](#).

Suite inventory

Suite	Location	Tests	Runtime	What it covers
Operators unit	tests/Pam.Operators.UnitTests/	18	~100ms	Brand aggregate behavior + CreateBrand validator
Identity unit	tests/Pam.Identity.UnitTests/	52	~1s	9 FluentValidation validators + PermissionResolver against EF Core in-memory
Players unit	tests/Pam.Players.UnitTests/	1	<100ms	Player.Create placeholder (scaffold)
Wallet unit	tests/Pam.Wallet.UnitTests/	1	<100ms	Account.Open placeholder (scaffold)
Architecture	tests/Pam.ArchitectureTests/	8	~250ms	NetArchTest rules: module boundary, domain/integration event shape, aggregate inheritance
Integration	tests/Pam.IntegrationTests/	3	~3s (incl. ~10–20s first-run container boot)	Health probes + auth fence over real Postgres/RabbitMQ/Redis via Testcontainers

Total: 83 tests. Architecture + unit suites finish in well under 2 seconds; integration is dominated by the one-time container boot.

Per-suite rationale

Unit tests (one project per module)

Purpose: lock down aggregate-level invariants and validator behavior. No DB, no broker, no DI graph — fast, deterministic, runnable on any machine without docker.

What's in scope:

- Aggregate factory methods + state transitions (e.g. Brand.Create stamps the slug, sets status to Active, raises BrandCreatedDomainEvent).
- FluentValidation rules — every input contract that hits a real endpoint has its validator covered (CreateBrand, CreateUser, ChangePassword, MfaVerify, ForgotPassword, ResetPassword, ConfirmEmail, LoginRecoveryCode, MfaDisable, ListUsers).
- PermissionResolver against the EF Core in-memory provider — the only place we use in-memory EF, because the resolver's job is to walk a join table, and that's relational shape that benefits from a real querying provider even if it isn't Postgres.

Out of scope:

- Anything that requires `DbContext.SaveChangesAsync` to commit to a real database. Those live in integration tests.
- Anything that requires the bus to actually accept a publish. Same.
- `Players` and `Wallet` aggregate behavior beyond `.Create` — those modules are scaffold-only; behavior tests land alongside the first features.

Conventions:

- xUnit v3 (`xunit.v3` package, NOT `xunit v2` — the v3 runner is in `xunit.runner.visualstudio 3.x`; never mix v2 and v3).
- FluentAssertions **pinned at 7.2.0** — last Apache-2.0 release. v8+ moved to the Xceed commercial license. Do not bump without legal sign-off.
- No NSubstitute mocking of `DbContext` or `IPublishEndpoint`. Either the test doesn't need them (pure aggregate behavior) or it should graduate to integration.

Architecture tests (`Pam.ArchitectureTests`)

Purpose: encode the structural rules from [ARCHITECTURE.md](#) as automated checks so they survive refactoring pressure. Failed arch tests block the build the same way failed unit tests do.

What each rule encodes:

Test	Rule	Why it matters
Operators_Does_Not_Depend_On_Identity_Internals	Pam.Operators may not reference Pam.Identity directly	Cross-module coupling makes future microservice extraction expensive; *.Contracts is the only allowed seam
Operators_Does_Not_Depend_On_Notifications_Internals	same, Pam.Operators → Pam.Notifications	same
Identity_Does_Not_Depend_On_Operators_Internals	same, reverse direction	same
Notifications_Does_Not_Depend_On_Identity_Internals	same	same
Notifications_Does_Not_Depend_On_Operators_Internals	same	same
Domain_Events_Implement_IDomainEvent	types ending in DomainEvent implement IDomainEvent	The interceptor finds events by interface, not by name — but the naming convention is what makes a single grep find every event
Integration_Events_End_In_IntegrationEvent	types in *.IntegrationEvents.* end in IntegrationEvent	Same reasoning — naming is the lookup convention, this enforces it
Aggregates_Inherit_From_Aggregate_Or_Entity	concrete IAggregate implementers derive from Aggregate<>	The Aggregate<T> base class is what carries the DomainEvents list the dispatcher reads from

Out of scope for the gate (deliberately):

- Players and Wallet aren't yet exercised by the boundary rules. Add them when those modules grow real cross-module touchpoints — adding rules over empty modules is noise.
- Naming rules for handlers / endpoints / configurations. Convention is enforced by code review; arch tests would lock the patterns prematurely.

Note on TestResult: NetArchTest.Rules.TestResult and Xunit.TestResult collide. Both files alias the NetArchTest one: using ArchTestResult = NetArchTest.Rules.TestResult; .

Integration tests (Pam.IntegrationTests)

Purpose: prove the host actually boots and serves requests end-to-end against real infrastructure. Catches everything unit tests can't: EF migrations applying cleanly, the DI graph composing, real broker round-trips, real Identity flow, real HTTP pipeline ordering.

The harness:

- `PamContainersFixture : IAsyncLifetime` boots Postgres 17 + RabbitMQ 4 + Redis 7 via Testcontainers. Shared as an `ICollectionFixture` via `PamContainersCollection`, so the ~10–20s boot cost is paid once per test session, not per test class.
- `PamApiFactory : WebApplicationFactory<Program>` overrides `ConnectionStrings` and `MessageBroker` via `ConfigureAppConfiguration`. Services-collection overrides are too late — `Program.cs` awaits `ConnectionMultiplexer.ConnectAsync` at the top level **before** `builder.Build()`, so config has to be in place before the host is built.
- RabbitMQ credentials come from `containers.Rabbit.GetConnectionString()`, parsed for user/pass. Testcontainers rolls fresh non-guest credentials per container.
- A unique `DataProtection.ApplicationName` per `PamApiFactory` instance keeps DP keys isolated across parallel test runs.

Current coverage:

Test	Verifies
<code>LiveProbe_Returns_Healthy</code>	<code>GET /health/live</code> returns 200 — host is up, route is mapped
<code>ReadyProbe_Returns_Healthy_When_Dependencies_Up</code>	<code>GET /health/ready</code> returns 200 — Postgres + Redis + Rabbit all reachable. Uses <code>HealthCheckService</code> directly so failures report which dependency broke, not just "Unhealthy"
<code>CreateBrand_Requires_Authentication</code>	Anonymous <code>POST /v1/operators/brands</code> → 401. The fallback authorization policy is intact

Why these three, and not more:

- The point of the test gate is the *harness*, not coverage. Once the harness is in place, adding a test for any endpoint is mechanical.
- Authenticated end-to-end coverage (e.g. create-brand → get-brand round trip) needs a test-only auth seam to issue a dev bearer token. That seam isn't built yet — it's the next integration-test PR.

Conventions inside integration tests:

- xUnit v3 quirks: `TestContext.Current.CancellationToken` for any async call that accepts one (analyzer xUnit1051 enforces this).
- `HttpClient.GetAsync` takes a `Uri`, not a string (CA2234).
- New `PamApiFactory` per test method (`await using`) — sharing factories across tests would race the DP keyring writes and hosted-service shutdown.

Out of scope for the gate (deliberately):

- Performance / load tests. The infra is wired but a "1k req/s" baseline test needs realistic data and isn't useful before Wallet has real domain code.
- Migration rollback tests. EF migrations are forward-only; rollback is a manual process that runs against a real Postgres backup.
- Multi-replica integration test (boot two `PamApiFactory` instances against the same containers, verify cookie/token cross-replica validation). This is the natural follow-up to the pre-replica-safety PRs — useful, but not blocking.

What none of these suites cover yet

Listed here so the absence is intentional, not accidental:

- **Auth-aware integration tests.** As above — needs a test-only bearer seam. Most of the actual endpoint logic lives behind auth, so this is the biggest coverage gap.
- **Outbox transactional semantics.** The unit tests cover validators and aggregate behavior; the integration tests stop at health probes. A test that publishes a `BrandCreatedIntegrationEvent`, forces a `SaveChanges` failure, and asserts no `outbox_message` row was written would be the canonical assertion for the pre-save-dispatch + outbox combo. Lands when the first cross-module consumer ships in `Pam.Notifications`.
- **Multi-replica behavior.** Cookies issued by replica A validate on replica B (the whole point of PR #7's DP keys work). No test exercises this yet.
- **Rate limiter under concurrency.** The Redis-backed limiter enforces global counts; nothing tests the "6 requests across 2 replicas in 1 minute → 6th gets 429" case yet.
- **Players + Wallet beyond `.Create`.** Behavior tests land with the real features.
- **Mutation tests.** Out of scope until coverage is broader.

Running the tests

All suites

```
make test
# or, equivalently:
dotnet test pam.slnx --nologo
```

The integration suite requires Docker (or Podman with a Docker socket) running. Testcontainers will fail with a clear "Docker endpoint unreachable" error if it isn't — no other dev infrastructure is required (Testcontainers manages its own Postgres/Rabbit/Redis containers, separate from the long-lived ones in `docker-compose.yml`).

A specific suite

```
# unit only – Operators
dotnet test tests/Pam.Operators.UnitTests/Pam.Operators.UnitTests.csproj --nologo

# architecture rules
dotnet test tests/Pam.ArchitectureTests/Pam.ArchitectureTests.csproj --nologo

# integration only (needs Docker)
dotnet test tests/Pam.IntegrationTests/Pam.IntegrationTests.csproj --nologo
```

Filtering

```
# everything except integration (offline-friendly)
dotnet test pam.slnx --nologo --filter "FullyQualifiedName!~Pam.IntegrationTests"

# a single test method
dotnet test pam.slnx --nologo --filter "FullyQualifiedName~CreateBrand_Requires_Authentication"
```

When integration tests flake

The most common failure mode is "container didn't come up in time". Symptoms: `Timeouts`, `Broker unreachable`, `connection refused`. Order of things to check:

1. `docker ps` — is Docker actually running? Testcontainers needs a live daemon.
2. Disk space — Docker creates fresh containers per test run; a full disk surfaces as opaque container-start failures.
3. Resource limits — boot times stretch under memory pressure. The Rabbit container especially can take 15s+ to be reachable on a loaded laptop.

If `ReadyProbe_Returns_Healthy_When_Dependencies_Up` fails, the exception message names which health check returned non-Healthy and why — the test resolves `HealthCheckService` directly rather than relying on the default response writer's bare `"Unhealthy"` body, so you don't have to dig.

Conventions cheat-sheet

What	Rule	Lives where
Test framework	xUnit v3 only	<code>Directory.Packages.props</code> pins <code>xunit.v3 3.x</code> and <code>xunit.runner.visualstudio 3.x</code>
Assertions	FluentAssertions 7.2.0 (Apache-2.0 last)	Don't bump to 8.x — license change
Mocking	NSubstitute when needed; not for <code>DbContext</code> / <code>IPublishEndpoint</code>	—
Containers	Testcontainers 4.x	New API uses image-in-constructor (<code>new PostgreSQLBuilder("postgres:17")</code>) — the parameterless ctor + <code>.WithImage(...)</code> is obsolete
Cancellation in async tests	<code>TestContext.Current.CancellationToken</code>	xUnit1051 analyzer enforces
<code>HttpClient.GetAsync</code>	Pass <code>Uri</code> , not <code>string</code>	CA2234 analyzer enforces
One factory per integration test	<code>await using var factory = new PamApiFactory(containers);</code>	Sharing trips DP keyring races
<code>Program</code> access from tests	<code>public partial class Program;</code> trailer in <code>Program.cs</code>	Required for <code>WebApplicationFactory<Program></code> to bind to the top-level-statement-synthesized type